Lattice-Planner 源程序鉴别材料（前 30 页 + 后 30 页）

> 说明：本文件按每页约 60 行的连续源程序行数，从完整程序清单中选取前 30 页和后 30 页，用于软件著作权登记的源程序鉴别材料参考。

前 30 页（约 1800 行）

```
// ===== File: src/main/java/org/zhzssp/memorandum/config/CorsConfig.j
package org.zhzssp.memorandum.config;

import org.jetbrains.annotations.NotNull;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigu

@Configuration
public class CorsConfig {
    @Bean
    public WebMvcConfigurer corsConfigurer() {
        return new WebMvcConfigurer() {
            @Override
            public void addCorsMappings(@NotNull CorsRegistry registry
                registry.addMapping("/**")
                        .allowedOriginPatterns("*") // Electron 所在源
                        .allowCredentials(true)
                        .allowedMethods("*");
            }
        };
    }
}
// ===== File: src/main/java/org/zhzssp/memorandum/config/MyBatisConfi
package org.zhzssp.memorandum.config;

import org.mybatis.spring.annotation.MapperScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@MapperScan("org.zhzssp.memorandum.mapper")
public class MyBatisConfig {
}
```

```
// ===== File: src/main/java/org/zhzssp/memorandum/config/NoCacheConfi
package org.zhzssp.memorandum.config;

import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.HandlerInterceptor;
import org.springframework.web.servlet.config.annotation.InterceptorRe
import org.springframework.web.servlet.config.annotation.WebMvcConfigu

@Configuration
public class NoCacheConfig implements WebMvcConfigurer {

    private static class NoCacheInterceptor implements HandlerIntercep
        @Override
        public boolean preHandle(HttpServletRequest request, HttpServl
            String uri = request.getRequestURI();
            if (uri.equals("/login") || uri.equals("/select-features")
                response.setHeader("Cache-Control", "no-cache, no-stor
                response.setHeader("Pragma", "no-cache");
                response.setDateHeader("Expires", 0);
            }
            return true;
        }
    }

    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(new NoCacheInterceptor())
                .addPathPatterns("/login", "/select-features");
    }
}
// ===== File: src/main/java/org/zhzssp/memorandum/config/WebSecurityC
package org.zhzssp.memorandum.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Bean;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.config.annotation.web.configuratio

import org.springframework.security.core.context.SecurityContextHolder
import org.zhzssp.memorandum.service.CustomUserDetailsService;
import org.springframework.security.config.annotation.web.builders.Htt
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncode
import org.springframework.security.web.SecurityFilterChain;

@EnableWebSecurity
```

```
@Configuration
public class WebSecurityConfig {

    @Autowired
    private CustomUserDetailsService userDetailsService;

    @Bean
    public BCryptPasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws E
        http
                // 保持CSRF保护，但为API端点配置豁免
                .csrf(csrf -> csrf
                        .ignoringRequestMatchers("/user-logged-in", "/
                )
                .authorizeHttpRequests(auth -> auth
                        // 允许访问根路径与登录、注册及静态资源
                        .requestMatchers("/", "/register", "/login", "
                        .anyRequest().authenticated()
                )
                .formLogin(login -> login
                        .loginPage("/login")
                        .defaultSuccessUrl("/select-features", true) /
                        .permitAll()
                )
                .logout(logout -> logout.permitAll());
        return http.build();
    }
}

// ===== File: src/main/java/org/zhzssp/memorandum/controller/AuthCont
package org.zhzssp.memorandum.controller;

import org.zhzssp.memorandum.entity.User;
import org.zhzssp.memorandum.repository.UserRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncode
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.*;
import org.springframework.ui.Model;
import org.springframework.security.core.context.SecurityContextHolder
import org.springframework.security.core.Authentication;

// 没有使用ResponseBody，表示该类的方法返回的是视图名称（模板名称）
```

```
// 若使用ResponseBody / RestController，则表示返回的是封装在HTTP相应之中
@Controller
public class AuthController {
    @Autowired
    private UserRepository userRepository;

    @GetMapping("/register")
    public String registerForm() {
        return "register"; // 渲染register.html模板
    }

    // 不使用"/"时，login/register或addMemo/register等情况会导致无法正确
    // 接收POST /register请求（内容包括user, password）,处理用户注册
    @PostMapping("/register")
    public String register(@RequestParam String username, @RequestPara
        // 后端验证密码长度
        if (password.length() < 5) {
            // 密码不符合要求，返回注册页面并显示错误消息
            model.addAttribute("errorMessage", "Password must be at le
            // 使用redirect会发出GET请求，导致错误信息无法显示
            return "register";
        }

        User user = new User();
        user.setUsername(username);
        user.setPassword(new BCryptPasswordEncoder().encode(password))
        // 调用JPA自动生成的SQL语句，保存用户到数据库
        userRepository.save(user);
        // 注册成功后重定向到login页面
        return "redirect:/login";
    }

    /**
     * 登录页：
     * - 默认保持原有逻辑：若已登录且已选择模式，则直接跳转 dashboard；
     * - 当带上 ?force=true 时，强制清理登录状态和会话，回到登录界面。
     */
    @GetMapping("/login")
    public String loginForm(
            @RequestParam(name = "force", required = false, defaultVal
            jakarta.servlet.http.HttpSession session) {
        Authentication auth = SecurityContextHolder.getContext().getAu

        // 强制重新登录：清理认证与会话
        if (force) {
            if (auth != null && auth.isAuthenticated() && !"anonymousU
                SecurityContextHolder.clearContext();
```

```
        }
        if (session != null) {
            session.invalidate();
        }
        return "login";
    }

    // 原有逻辑：已登录且已选择模式 -> 跳 dashboard
    boolean loggedIn = auth != null && auth.isAuthenticated() && !
    boolean modeChosen = session != null && session.getAttribute("
    if (loggedIn && modeChosen) {
        return "redirect:/dashboard";
    }
    return "login";
}

/**
 * 访问根路径时统一跳转到登录页，并强制重新登录。
 * 这样在浏览器地址栏输入 http://localhost:8080 时：
 * - 不会出现 Whitelabel Error Page；
 * - 且用户会被要求重新输入账号密码。
 */
@GetMapping("/")
public String root() {
    return "redirect:/login?force=true";
}

@GetMapping("/user-logged-in")
// 返回值为数据而非视图名称
@ResponseBody
public boolean sendUserLoggedInNotification() {
    try {
        Authentication authentication = SecurityContextHolder.getC

        System.out.println("=== Authentication Debug Info ===");
        System.out.println("Authentication object: " + authenticat

        if (authentication == null) {
            System.out.println("Authentication is NULL");
            return false;
        }

        boolean isAuthenticated = authentication.isAuthenticated()
        String username = authentication.getName();
        String authType = authentication.getClass().getSimpleName(

        // 出现anonymousUser的情况?
```

```
            System.out.println("Authentication type: " + authType);
            System.out.println("Is authenticated: " + isAuthenticated)
            System.out.println("Username: " + username);
            System.out.println("Is anonymous: " + username.equals("ano
            System.out.println("Authorities: " + authentication.getAut
            System.out.println("================================");

            return isAuthenticated && !username.equals("anonymousUser"
            // return isAuthenticated;
        } catch (Exception e) {
            System.out.println("Error in authentication check: " + e.g
            e.printStackTrace();
            return false;
        }
    }
}
// ===== File: src/main/java/org/zhzssp/memorandum/controller/FeatureS
package org.zhzssp.memorandum.controller;

import jakarta.servlet.http.HttpSession;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.zhzssp.memorandum.entity.User;
import org.zhzssp.memorandum.entity.UserPreference;
import org.zhzssp.memorandum.repository.UserRepository;
import org.zhzssp.memorandum.service.UserPreferenceService;

import java.security.Principal;

@Controller
public class FeatureSelectionController {

    private static final String SESSION_KEY = "mindsetMode"; // 思维模

    @Autowired
    private UserRepository userRepository;

    @Autowired
    private UserPreferenceService userPreferenceService;

    @GetMapping("/select-features")
    public String selectFeaturesPage(jakarta.servlet.http.HttpSession
        // 若已经选择过思维模式则直接跳转至 dashboard，防止回退再访问
```

```
        if (session.getAttribute(SESSION_KEY) != null) {
            return "redirect:/dashboard";
        }
        if (principal != null) {
            User user = userRepository.findByUsername(principal.getNam
            if (user != null) {
                UserPreference pref = userPreferenceService.getOrCreat
                model.addAttribute("defaultMindsetMode", pref.getDefau
            }
        }
        return "selectFeatures";
    }

    @PostMapping("/select-features")
    public String saveMindsetMode(@RequestParam(name = "mindsetMode",
                                  HttpSession session) {
        // default to execute if nothing sent
        String validMode = mode != null && (mode.equals("execute") ||
                ? mode : "execute";
        session.setAttribute(SESSION_KEY, validMode);
        return "redirect:/dashboard";
    }

    public static String getMindsetMode(HttpSession session) {
        Object obj = session.getAttribute(SESSION_KEY);
        return obj instanceof String ? (String) obj : "execute";
    }

    /**
     * 供 Dashboard 顶部的「切换视图模式」按钮使用：
     * 清除当前会话中的思维模式，下次进入重新选择。
     */
    @GetMapping("/change-mode")
    public String changeMode(HttpSession session) {
        session.removeAttribute(SESSION_KEY);
        return "redirect:/select-features";
    }
}
// ===== File: src/main/java/org/zhzssp/memorandum/controller/GoalTask
package org.zhzssp.memorandum.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import org.zhzssp.memorandum.entity.Task;
import org.zhzssp.memorandum.entity.User;
```

```
import org.zhzssp.memorandum.feature.goal.dto.GoalWithTasks;
import org.zhzssp.memorandum.feature.goal.dto.TreeNodeDto;
import org.zhzssp.memorandum.feature.goal.service.GoalService;
import org.zhzssp.memorandum.repository.UserRepository;

import java.security.Principal;
import java.time.format.DateTimeFormatter;
import java.util.List;
import java.util.stream.Collectors;

/**
 * 目标与任务树状图 API：返回层级 JSON 供前端 D3 绘制节点与边。
 */
@RestController
@RequestMapping("/api")
public class GoalTaskTreeApiController {

    private static final DateTimeFormatter DATE_FMT = DateTimeFormatte

    @Autowired
    private GoalService goalService;

    @Autowired
    private UserRepository userRepository;

    @GetMapping("/goal-task-tree")
    public TreeNodeDto getGoalTaskTree(Principal principal) {
        User user = userRepository.findByUsername(principal.getName())
        List<GoalWithTasks> list = goalService.findGoalTaskTree(user);
        List<TreeNodeDto> goalNodes = list.stream()
                .map(item -> new TreeNodeDto(
                        item.goal().getName(),
                        "goal",
                        item.goal().getId(),
                        item.goal().getGoalType() != null ? item.goal(
                        null,
                        item.tasks().stream()
                                .map(t -> new TreeNodeDto(
                                        t.getTitle(),
                                        "task",
                                        t.getId(),
                                        null,
                                        t.getDeadline() != null ? t.ge
                                        null
                                ))
                                .collect(Collectors.toList())
                ))
```

```
                .collect(Collectors.toList());
        return new TreeNodeDto("根", "root", null, null, null, goalNod
    }
}
// ===== File: src/main/java/org/zhzssp/memorandum/controller/NoteCont
package org.zhzssp.memorandum.controller;

import lombok.Data;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.util.StringUtils;
import org.springframework.web.bind.annotation.*;
import org.zhzssp.memorandum.entity.Note;
import org.zhzssp.memorandum.entity.NoteType;
import org.zhzssp.memorandum.entity.User;
import org.zhzssp.memorandum.repository.NoteRepository;
import org.zhzssp.memorandum.repository.UserRepository;

import java.security.Principal;
import java.util.List;

@RestController
@RequestMapping("/note")
public class NoteController {

    @Autowired
    private NoteRepository noteRepository;

    @Autowired
    private UserRepository userRepository;

    @GetMapping("/list")
    public List<Note> listNotes(Principal principal) {
        User user = userRepository.findByUsername(principal.getName())
        return noteRepository.findByUser(user);
    }

    @PostMapping("/add")
    public ResponseEntity<?> addNote(@RequestBody NewNoteDto dto, Prin
        if (!StringUtils.hasText(dto.getTitle()) && !StringUtils.hasTe
            return ResponseEntity.badRequest().body("empty");
        }
        User user = userRepository.findByUsername(principal.getName())
        Note note = new Note();
        note.setTitle(dto.getTitle());
        note.setContent(dto.getContent());
```

```
        NoteType type = NoteType.SCRATCH;
        if (StringUtils.hasText(dto.getType())) {
            try {
                type = NoteType.valueOf(dto.getType());
            } catch (IllegalArgumentException ignored) {
            }
        }
        note.setType(type);

        note.setUser(user);
        noteRepository.save(note);
        return ResponseEntity.ok(note.getId());
    }

    @Data
    public static class NewNoteDto {
        private String title;
        private String content;
        private String type;
    }
}
// ===== File: src/main/java/org/zhzssp/memorandum/controller/TaskCont
package org.zhzssp.memorandum.controller;

import org.jetbrains.annotations.NotNull;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.util.StringUtils;
import org.springframework.web.bind.annotation.*;
import org.zhzssp.memorandum.entity.*;
import org.zhzssp.memorandum.repository.NoteRepository;
import org.zhzssp.memorandum.repository.TaskRepository;
import org.zhzssp.memorandum.repository.UserRepository;
import org.zhzssp.memorandum.core.service.TaskService;
import org.zhzssp.memorandum.feature.goal.dto.GoalWithTasks;
import org.zhzssp.memorandum.feature.goal.entity.Goal;
import org.zhzssp.memorandum.feature.goal.service.GoalService;
import org.zhzssp.memorandum.service.UserPreferenceService;
import org.zhzssp.memorandum.entity.UserPreference;
import org.zhzssp.memorandum.entity.Note;

import java.util.Map;
import java.util.HashMap;

import java.security.Principal;
import java.time.LocalDate;
```

```
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeFormatterBuilder;
import java.util.List;
import java.util.stream.Collectors;

/**
 * 任务相关操作。统一使用 Task 概念，URL 保留 /memo/* 以兼容现有书签。
 */
@Controller
public class TaskController {

    @Autowired
    private TaskRepository taskRepository;

    @Autowired
    private UserRepository userRepository;

    @Autowired
    private TaskService taskService;

    @Autowired
    private GoalService goalService;

    @Autowired
    private UserPreferenceService userPreferenceService;

    @Autowired
    private NoteRepository noteRepository;

    @GetMapping("/dashboard")
    public String dashboard(@NotNull Model model,
                            Principal principal,
                            @RequestParam(name = "taskView", required
                            jakarta.servlet.http.HttpSession session)
        User user = userRepository.findByUsername(principal.getName())
        List<Task> allTasks = taskRepository.findByUser(user);

        // 思维模式：execute/learn/plan
        String mindsetMode = FeatureSelectionController.getMindsetMode
        // 兼容旧 session：tasks/notes 映射到 execute/learn
        if ("tasks".equals(mindsetMode)) {
            mindsetMode = "execute";
        } else if ("notes".equals(mindsetMode)) {
            mindsetMode = "learn";
        } else if (!mindsetMode.equals("execute") && !mindsetMode.equa
            mindsetMode = "execute";
```

```
}
model.addAttribute("mindsetMode", mindsetMode);

// 获取用户偏好
UserPreference preference = userPreferenceService.getOrCreateP
model.addAttribute("preference", preference);
model.addAttribute("theme", preference.getTheme() != null ? pr
model.addAttribute("goalTaskTree", goalService.findGoalTaskTre

// 规划模式下使用用户偏好的默认视图；非规划模式固定为今日
if ("plan".equals(mindsetMode)) {
    if (taskView == null || taskView.isEmpty() || !List.of("to
        taskView = (preference.getDefaultTaskView() != null &&
                ? preference.getDefaultTaskView() : "today";
    }
} else {
    taskView = "today";
}
model.addAttribute("taskView", taskView);

// 今日视图的基础任务集合（PENDING + deadline=今天或未设置）
List<Task> todayTasks = taskService.getTodayActionableTasks(us

// 根据思维模式应用不同的数据准备逻辑
if ("execute".equals(mindsetMode)) {
    // 执行模式：低信息密度，只看今日+当前任务
    // 应用用户偏好过滤
    List<Task> filteredTasks = userPreferenceService.applyTask
    model.addAttribute("tasks", filteredTasks);
    model.addAttribute("memos", filteredTasks); // 兼容模板

    // 不显示目标管理、统计等复杂信息
    model.addAttribute("showGoals", false);
    model.addAttribute("showStatistics", false);

} else if ("learn".equals(mindsetMode)) {
    // 学习模式：中信息密度，任务+笔记并排
    List<Task> filteredTasks = userPreferenceService.applyTask
    model.addAttribute("tasks", filteredTasks);
    model.addAttribute("memos", filteredTasks);

    // 获取笔记列表
    List<Note> notes = noteRepository.findByUser(user);
    model.addAttribute("notes", notes);

    // 显示笔记创建功能，但不显示目标管理
    model.addAttribute("showGoals", false);
```

```
        model.addAttribute("showStatistics", false);

    } else if ("plan".equals(mindsetMode)) {
        // 规划模式：高信息密度，目标/项目/全局
        // 显示所有任务（包括未来任务，如果用户偏好允许）
        List<Task> allPendingTasks = allTasks.stream()
                .filter(t -> t.getEffectiveStatus() == TaskStatus.
                .collect(Collectors.toList());
        List<Task> filteredTasks = userPreferenceService.applyTask

        // 时间段视图分组（如果 taskView=slot）
        if ("slot".equals(taskView)) {
            List<Task> morningTasks = filteredTasks.stream()
                    .filter(t -> t.getPreferredSlot() == TimeSlot.
                    .collect(Collectors.toList());
            List<Task> afternoonTasks = filteredTasks.stream()
                    .filter(t -> t.getPreferredSlot() == TimeSlot.
                    .collect(Collectors.toList());
            List<Task> eveningTasks = filteredTasks.stream()
                    .filter(t -> t.getPreferredSlot() == TimeSlot.
                    .collect(Collectors.toList());
            List<Task> otherSlotTasks = filteredTasks.stream()
                    .filter(t -> t.getPreferredSlot() == null)
                    .collect(Collectors.toList());
            model.addAttribute("morningTasks", morningTasks);
            model.addAttribute("afternoonTasks", afternoonTasks);
            model.addAttribute("eveningTasks", eveningTasks);
            model.addAttribute("otherSlotTasks", otherSlotTasks);
        } else if ("energy".equals(taskView)) {
            // 精力视图：按精力需求分组
            List<Task> highEnergyTasks = filteredTasks.stream()
                    .filter(t -> t.getEnergyRequirement() == Energ
                    .collect(Collectors.toList());
            List<Task> mediumEnergyTasks = filteredTasks.stream()
                    .filter(t -> t.getEnergyRequirement() == Energ
                    .collect(Collectors.toList());
            List<Task> lowEnergyTasks = filteredTasks.stream()
                    .filter(t -> t.getEnergyRequirement() == Energ
                    .collect(Collectors.toList());
            List<Task> otherEnergyTasks = filteredTasks.stream()
                    .filter(t -> t.getEnergyRequirement() == null)
                    .collect(Collectors.toList());
            model.addAttribute("highEnergyTasks", highEnergyTasks)
            model.addAttribute("mediumEnergyTasks", mediumEnergyTa
            model.addAttribute("lowEnergyTasks", lowEnergyTasks);
            model.addAttribute("otherEnergyTasks", otherEnergyTask
        } else {
```

```
        // 今日视图：单列表展示
        model.addAttribute("tasks", filteredTasks);
        model.addAttribute("memos", filteredTasks);
    }

    // 显示目标管理、统计等（可由偏好关闭）
    model.addAttribute("showGoals", preference.getShowGoals()
    model.addAttribute("showScoreSection", preference.getShowS
    model.addAttribute("showStatistics", preference.getShowSta

    // 计算统计信息（如果用户偏好允许）
    if (preference.getShowStatistics()) {
        long totalTasks = allTasks.size();
        long doneTasks = allTasks.stream()
                .filter(t -> t.getEffectiveStatus() == TaskSta
                .count();
        double completionRate = totalTasks > 0 ? (doneTasks *
        model.addAttribute("totalTasks", totalTasks);
        model.addAttribute("doneTasks", doneTasks);
        model.addAttribute("completionRate", String.format("%.
    }
}

// 任务→目标弱关联（用于展示标签）- 仅在需要时计算
Map<Long, List<Goal>> taskToGoals = new HashMap<>();
List<Task> tasksForGoals = "plan".equals(mindsetMode) || "lear
        ? (model.getAttribute("tasks") != null ? (List<Task>)
        : todayTasks;
for (Task t : tasksForGoals) {
    taskToGoals.put(t.getId(), goalService.findGoalsByTaskId(t
}
model.addAttribute("taskToGoals", taskToGoals);

// 仅在规划模式下显示目标管理
if ("plan".equals(mindsetMode)) {
    model.addAttribute("goals", goalService.findActiveGoalsByU
}

// 模糊任务 N 天存在提示（仅在规划模式且偏好允许时显示）
if ("plan".equals(mindsetMode) && preference.getShowFuzzyHint(
    int fuzzyDays = preference.getFuzzyTaskDaysThreshold() !=
            ? preference.getFuzzyTaskDaysThreshold() : 5;
    if (fuzzyDays > 0) {
        List<Task> fuzzyNeedSplit = taskService.findFuzzyTasks
        model.addAttribute("fuzzyNeedSplit", fuzzyNeedSplit);
    } else {
        model.addAttribute("fuzzyNeedSplit", List.of());
```

```
        }
    } else if ("plan".equals(mindsetMode)) {
        model.addAttribute("fuzzyNeedSplit", List.of());
    }

    // 已完成 / 归档 / 搁置任务，仅在规划模式且偏好允许时折叠展示
    if ("plan".equals(mindsetMode) && preference.getShowArchivedSe
        List<Task> archivedOrShelved = allTasks.stream()
                .filter(t -> t.getEffectiveStatus() != TaskStatus.
                .collect(Collectors.toList());
        model.addAttribute("archivedOrShelved", archivedOrShelved)
    } else if ("plan".equals(mindsetMode)) {
        model.addAttribute("archivedOrShelved", List.of());
    }

    return "dashboard";
}

@GetMapping("/memo/search")
public String searchTasks(@RequestParam(required = false) String k
                          @RequestParam(required = false, name = "s
                          @RequestParam(required = false, name = "e
                          Principal principal,
                          Model model,
                          jakarta.servlet.http.HttpSession session)
    if (!StringUtils.hasText(keyword) && !StringUtils.hasText(star
        return "redirect:/dashboard";
    }

    User user = userRepository.findByUsername(principal.getName())

    LocalDateTime start = null;
    LocalDateTime end = null;
    DateTimeFormatter dateFmt = DateTimeFormatter.ofPattern("yyyy-
    try {
        if (StringUtils.hasText(startDate)) {
            start = LocalDate.parse(startDate, dateFmt).atStartOfD
        }
        if (StringUtils.hasText(endDate)) {
            end = LocalDate.parse(endDate, dateFmt).atTime(23, 59,
        }
    } catch (Exception e) {
        // ignore
    }

    List<Task> tasks = taskService.searchTasks(user.getId(), keywo
    model.addAttribute("tasks", tasks);
```

```
    model.addAttribute("memos", tasks);
    model.addAttribute("mode", "tasks");

    Map<Long, List<Goal>> taskToGoals = new HashMap<>();
    for (Task t : tasks) {
        taskToGoals.put(t.getId(), goalService.findGoalsByTaskId(t
    }
    model.addAttribute("taskToGoals", taskToGoals);
    model.addAttribute("goals", goalService.findActiveGoalsByUser(

    model.addAttribute("dueSoonTasks", List.of());
    model.addAttribute("dueSoonMemos", List.of());
    UserPreference pref = userPreferenceService.getOrCreatePrefere
    model.addAttribute("theme", pref.getTheme() != null ? pref.get
    model.addAttribute("goalTaskTree", goalService.findGoalTaskTre
    return "dashboard";
}

@GetMapping("/memo/add")
public String addTaskForm(Principal principal, Model model) {
    User user = userRepository.findByUsername(principal.getName())
    model.addAttribute("goals", goalService.findActiveGoalsByUser(
    return "addMemo";
}

@PostMapping("/memo/add")
public String addTask(@RequestParam String title,
                      @RequestParam String description,
                      @RequestParam String deadline,
                      @RequestParam(required = false) String granul
                      @RequestParam(required = false) Integer estim
                      @RequestParam(required = false) String energy
                      @RequestParam(required = false) String mental
                      @RequestParam(required = false) String prefer
                      @RequestParam(required = false) List<Long> go
                      Principal principal) {
    User user = userRepository.findByUsername(principal.getName())
    Task task = new Task();
    task.setTitle(title);
    task.setDescription(description);
    DateTimeFormatter formatter = new DateTimeFormatterBuilder()
            .appendPattern("yyyy-MM-dd'T'HH:mm")
            .optionalStart()
            .appendPattern(":ss")
            .optionalEnd()
            .toFormatter();
    task.setDeadline(LocalDateTime.parse(deadline, formatter));
```

```
    if (granularity != null && !granularity.isEmpty()) {
        try {
            task.setGranularity(TaskGranularity.valueOf(granularit
        } catch (IllegalArgumentException ignored) {
        }
    }
    task.setEstimatedMinutes(estimatedMinutes);
    if (energyRequirement != null && !energyRequirement.isEmpty())
        try {
            task.setEnergyRequirement(EnergyLevel.valueOf(energyRe
        } catch (IllegalArgumentException ignored) {
        }
    }
    if (mentalLoad != null && !mentalLoad.isEmpty()) {
        try {
            task.setMentalLoad(MentalLoad.valueOf(mentalLoad));
        } catch (IllegalArgumentException ignored) {
        }
    }
    if (preferredSlot != null && !preferredSlot.isEmpty()) {
        try {
            task.setPreferredSlot(TimeSlot.valueOf(preferredSlot))
        } catch (IllegalArgumentException ignored) {
        }
    }
    task.setStatus(TaskStatus.PENDING);
    taskService.saveTask(task, user);
    goalService.linkTaskToGoals(task.getId(), goalIds != null ? go
    return "redirect:/dashboard";
}

@GetMapping("/due-dates")
@ResponseBody
public List<Task> viewDueDates(Principal principal) {
    if (principal == null) {
        return List.of();
    }
    User user = userRepository.findByUsername(principal.getName())
    return taskRepository.findByUser(user);
}

@DeleteMapping("/memo/delete/{id}")
@ResponseBody
public String deleteTask(@PathVariable Long id, Principal principa
    try {
        User user = userRepository.findByUsername(principal.getNam
        Task task = taskRepository.findById(id).orElseThrow();
```

```
            if (!task.getUser().getId().equals(user.getId())) {
                return "error:unauthorized";
            }
            taskRepository.delete(task);
            return "success";
        } catch (Exception e) {
            return "error:failed";
        }
    }

    /** 完成任务（静默，不惩罚） */
    @PostMapping("/memo/complete/{id}")
    @ResponseBody
    public String completeTask(@PathVariable Long id, Principal princi
        try {
            User user = userRepository.findByUsername(principal.getNam
            Task task = taskRepository.findById(id).orElseThrow();
            if (!task.getUser().getId().equals(user.getId())) {
                return "error:unauthorized";
            }
            taskService.completeTask(task, user);
            return "success";
        } catch (Exception e) {
            return "error:failed";
        }
    }

    /** 搁置任务（静默搁置，不显示在默认视图） */
    @PostMapping("/memo/shelve/{id}")
    @ResponseBody
    public String shelveTask(@PathVariable Long id, Principal principa
        try {
            User user = userRepository.findByUsername(principal.getNam
            Task task = taskRepository.findById(id).orElseThrow();
            if (!task.getUser().getId().equals(user.getId())) {
                return "error:unauthorized";
            }
            task.setStatus(TaskStatus.SHELVED);
            task.setShelvedAt(LocalDateTime.now());
            taskRepository.save(task);
            return "success";
        } catch (Exception e) {
            return "error:failed";
        }
    }
}
// ===== File: src/main/java/org/zhzssp/memorandum/controller/UserPref
```

```
package org.zhzssp.memorandum.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;
import org.zhzssp.memorandum.entity.User;
import org.zhzssp.memorandum.entity.UserPreference;
import org.zhzssp.memorandum.repository.UserRepository;
import org.zhzssp.memorandum.service.UserPreferenceService;

import java.security.Principal;

@Controller
@RequestMapping("/preference")
public class UserPreferenceController {

    @Autowired
    private UserPreferenceService preferenceService;

    @Autowired
    private UserRepository userRepository;

    /** 可选：?mode=execute|learn|plan 用于默认选中的思维模式 tab */
    @GetMapping("/settings")
    public String settingsPage(Principal principal,
                               @RequestParam(name = "mode", required =
                               Model model) {
        User user = userRepository.findByUsername(principal.getName())
        UserPreference pref = preferenceService.getOrCreatePreference(
        model.addAttribute("preference", pref);
        model.addAttribute("theme", pref.getTheme() != null ? pref.get
        if (mode != null && (mode.equals("execute") || mode.equals("le
            model.addAttribute("selectedMode", mode);
        } else {
            model.addAttribute("selectedMode", "execute");
        }
        return "preferenceSettings";
    }

    @PostMapping("/update")
    public String updatePreference(@RequestParam(required = false) Int
                                   @RequestParam(required = false) Boo
                                   @RequestParam(required = false) Boo
                                   @RequestParam(required = false) Str
                                   @RequestParam(required = false) Str
                                   @RequestParam(required = false) Str
```

```
                            @RequestParam(required = false) Int
                            @RequestParam(required = false) Str
                            @RequestParam(required = false) Str
                            @RequestParam(required = false) Str
                            @RequestParam(required = false) Str
                            Principal principal) {
User user = userRepository.findByUsername(principal.getName())
UserPreference pref = preferenceService.getOrCreatePreference(

if (maxVisibleTasks != null && maxVisibleTasks > 0) {
    pref.setMaxVisibleTasks(maxVisibleTasks);
} else {
    pref.setMaxVisibleTasks(null);
}

// 未勾选的 checkbox 不会出现在 POST 中，为 null，必须显式视为 fa
pref.setShowFutureTasks(Boolean.TRUE.equals(showFutureTasks));
pref.setShowStatistics(Boolean.TRUE.equals(showStatistics));

if (defaultMindsetMode != null && !defaultMindsetMode.isBlank(
    try {
        pref.setDefaultMindsetMode(UserPreference.MindsetMode.
    } catch (IllegalArgumentException e) {
        pref.setDefaultMindsetMode(null);
    }
} else {
    pref.setDefaultMindsetMode(null);
}

if (theme != null && (theme.equals("light") || theme.equals("d
    pref.setTheme(theme);
} else {
    pref.setTheme(null);
}

if (defaultTaskView != null && (defaultTaskView.equals("slot")
    pref.setDefaultTaskView(defaultTaskView);
} else {
    pref.setDefaultTaskView(null);
}

if (fuzzyTaskDaysThreshold != null && fuzzyTaskDaysThreshold >
    pref.setFuzzyTaskDaysThreshold(fuzzyTaskDaysThreshold);
} else {
    pref.setFuzzyTaskDaysThreshold(null);
}
```

```
        // 同上：未勾选时参数不提交为 null，"true".equalsIgnoreCase(null
        pref.setShowGoals(showGoals != null && "true".equalsIgnoreCase
        pref.setShowFuzzyHint(showFuzzyHint != null && "true".equalsIg
        pref.setShowArchivedSection(showArchivedSection != null && "tr
        pref.setShowScoreSection(showScoreSection != null && "true".eq

        preferenceService.savePreference(pref);
        return "redirect:/dashboard";
    }

}
// ===== File: src/main/java/org/zhzssp/memorandum/core/event/TaskArch
package org.zhzssp.memorandum.core.event;

import org.zhzssp.memorandum.entity.Task;
import org.zhzssp.memorandum.entity.User;

/**
 * 任务归档事件。用于插件扩展，如统计更新等。
 */
public class TaskArchivedEvent {
    private final Task task;
    private final User user;

    public TaskArchivedEvent(Task task, User user) {
        this.task = task;
        this.user = user;
    }

    public Task getTask() {
        return task;
    }

    public User getUser() {
        return user;
    }
}
// ===== File: src/main/java/org/zhzssp/memorandum/core/event/TaskComp
package org.zhzssp.memorandum.core.event;

import org.zhzssp.memorandum.entity.Task;
import org.zhzssp.memorandum.entity.User;

/**
 * 任务完成事件。用于插件扩展，如统计更新、笔记推荐等。
 */
public class TaskCompletedEvent {
```

```
    private final Task task;
    private final User user;

    public TaskCompletedEvent(Task task, User user) {
        this.task = task;
        this.user = user;
    }

    public Task getTask() {
        return task;
    }

    public User getUser() {
        return user;
    }
}
// ===== File: src/main/java/org/zhzssp/memorandum/core/event/TaskCrea
package org.zhzssp.memorandum.core.event;

import org.zhzssp.memorandum.entity.Task;
import org.zhzssp.memorandum.entity.User;

/**
 * 任务创建事件。用于插件扩展，如目标聚类、任务分析等。
 */
public class TaskCreatedEvent {
    private final Task task;
    private final User user;

    public TaskCreatedEvent(Task task, User user) {
        this.task = task;
        this.user = user;
    }

    public Task getTask() {
        return task;
    }

    public User getUser() {
        return user;
    }
}
// ===== File: src/main/java/org/zhzssp/memorandum/core/service/TaskSe
package org.zhzssp.memorandum.core.service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.ApplicationEventPublisher;
```

```
import org.springframework.stereotype.Service;
import org.zhzssp.memorandum.core.event.TaskArchivedEvent;
import org.zhzssp.memorandum.core.event.TaskCompletedEvent;
import org.zhzssp.memorandum.core.event.TaskCreatedEvent;
import org.zhzssp.memorandum.entity.*;
import org.zhzssp.memorandum.mapper.TaskMapper;
import org.zhzssp.memorandum.repository.TaskRepository;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.Comparator;
import java.util.List;

/**
 * 核心任务服务。负责核心业务逻辑，通过事件机制与插件解耦。
 */
@Service
public class TaskService {

    @Autowired
    private TaskMapper taskMapper;

    @Autowired
    private TaskRepository taskRepository;

    @Autowired
    private ApplicationEventPublisher eventPublisher;

    public List<Task> searchTasks(Long userId, String keyword, LocalDa
        return taskMapper.searchTasks(userId, keyword, start, end);
    }

    /**
     * 查找「需要拆分提示」的任务：模糊 + 待办 + 创建时间早于 now - days。
     */
    public List<Task> findFuzzyTasksNeedingSplit(User user, long days)
        LocalDateTime threshold = LocalDateTime.now().minusDays(days);
        return taskRepository.findByUserAndGranularityAndStatusAndCrea
                user,
                TaskGranularity.FUZZY,
                TaskStatus.PENDING,
                threshold
        );
    }

    /**
     * 今日可行动任务：状态 PENDING，且 deadline 为今天或未设置。
```

```
 */
public List<Task> getTodayActionableTasks(User user) {
    List<Task> all = taskRepository.findByUser(user);
    LocalDate today = LocalDate.now();
    return all.stream()
            .filter(t -> t.getEffectiveStatus() == TaskStatus.PEND
            .filter(t -> t.getDeadline() == null || t.getDeadline(
            .sorted(taskComparatorBySlotAndEnergy())
            .toList();
}

/**
 * 保存任务并发布创建事件。
 */
public Task saveTask(Task task, User user) {
    task.setUser(user);
    Task saved = taskRepository.save(task);
    eventPublisher.publishEvent(new TaskCreatedEvent(saved, user))
    return saved;
}

/**
 * 完成任务并发布完成事件。
 */
public Task completeTask(Task task, User user) {
    task.setStatus(TaskStatus.DONE);
    Task saved = taskRepository.save(task);
    eventPublisher.publishEvent(new TaskCompletedEvent(saved, user
    return saved;
}

/**
 * 归档任务并发布归档事件。
 */
public Task archiveTask(Task task, User user) {
    task.setStatus(TaskStatus.ARCHIVED);
    Task saved = taskRepository.save(task);
    eventPublisher.publishEvent(new TaskArchivedEvent(saved, user)
    return saved;
}

/**
 * 用于今日视图排序：先按时间段（上午/下午/晚上/未指定），再按精力需求
 */
private Comparator<Task> taskComparatorBySlotAndEnergy() {
    return Comparator
            .comparing((Task t) -> slotOrder(t.getPreferredSlot())
```

```
                .thenComparing(t -> energyOrder(t.getEnergyRequirement
    }

    private int slotOrder(TimeSlot slot) {
        if (slot == null) return 3;
        return switch (slot) {
            case MORNING -> 0;
            case AFTERNOON -> 1;
            case EVENING -> 2;
        };
    }

    private int energyOrder(EnergyLevel level) {
        if (level == null) return 3;
        return switch (level) {
            case HIGH -> 0;
            case MEDIUM -> 1;
            case LOW -> 2;
        };
    }
}
// ===== File: src/main/java/org/zhzssp/memorandum/entity/EnergyLevel.
package org.zhzssp.memorandum.entity;

/**
 * 任务所需精力水平：用于弱时间管理推荐。
 */
public enum EnergyLevel {
    HIGH,
    MEDIUM,
    LOW
}

// ===== File: src/main/java/org/zhzssp/memorandum/entity/GoalType.jav
package org.zhzssp.memorandum.entity;

/**
 * 目标类型：长期/中期/短期，弱约束，不强制用户选择。
 */
public enum GoalType {
    /** 长期（季度/学期） */
    LONG_TERM,
    /** 中期（项目） */
    MID_TERM,
    /** 短期（任务/备忘） */
    SHORT_TERM
}
```

```java
// ===== File: src/main/java/org/zhzssp/memorandum/entity/Link.java ==
package org.zhzssp.memorandum.entity;

import jakarta.persistence.*;
import lombok.Data;
import org.hibernate.annotations.CreationTimestamp;

import java.time.LocalDateTime;

/**
 * 弱关联表：Task ↔ Note、Task ↔ Goal 等多对多关系。
 * 核心模型极简，高级能力通过此表扩展。
 */
@Entity
@Table(name = "link")
@Data
public class Link {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    /** 源类型：TASK, NOTE */
    @Enumerated(EnumType.STRING)
    @Column(name = "source_type", nullable = false)
    private LinkSourceType sourceType;

    @Column(name = "source_id", nullable = false)
    private Long sourceId;

    /** 目标类型：TASK, NOTE, GOAL */
    @Enumerated(EnumType.STRING)
    @Column(name = "target_type", nullable = false)
    private LinkTargetType targetType;

    @Column(name = "target_id", nullable = false)
    private Long targetId;

    @CreationTimestamp
    @Column(name = "created_at")
    private LocalDateTime createdAt;

    public enum LinkSourceType { TASK, NOTE }
    public enum LinkTargetType { TASK, NOTE, GOAL }
}
// ===== File: src/main/java/org/zhzssp/memorandum/entity/MentalLoad.j
package org.zhzssp.memorandum.entity;
```

```
/**
 * 任务心理负担：主观上的“重 / 轻”。
 */
public enum MentalLoad {
    HEAVY,
    LIGHT
}

// ===== File: src/main/java/org/zhzssp/memorandum/entity/Note.java ==
package org.zhzssp.memorandum.entity;

import jakarta.persistence.*;
import java.time.LocalDateTime;
import lombok.Data;
import org.hibernate.annotations.CreationTimestamp;
import org.hibernate.annotations.UpdateTimestamp;

@Data
@Entity
@Table(name = "note")
public class Note {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;

    @Column(columnDefinition = "TEXT")
    private String content;

    @Enumerated(EnumType.STRING)
    @Column(name = "type", nullable = true)
    private NoteType type;

    @CreationTimestamp
    private LocalDateTime createdAt;

    @UpdateTimestamp
    private LocalDateTime updatedAt;

    @ManyToOne
    private User user;
}
// ===== File: src/main/java/org/zhzssp/memorandum/entity/NoteType.jav
package org.zhzssp.memorandum.entity;
```

```java
/**
 * 笔记类型：用于区分结构与展示。
 */
public enum NoteType {
    SCRATCH,        // 临时想法
    LEARNING,       // 学习笔记
    PROJECT,        // 项目笔记
    RETROSPECTIVE   // 复盘笔记
}

// ===== File: src/main/java/org/zhzssp/memorandum/entity/Task.java ==
package org.zhzssp.memorandum.entity;

import jakarta.persistence.*;
import lombok.Data;
import org.hibernate.annotations.CreationTimestamp;

import java.time.LocalDateTime;

/**
 * 核心任务实体。原 Memo 概念统一为 Task，为扩展预留空间。
 * 表名暂保留 memo 以兼容现有数据。
 */
@Entity
@Table(name = "memo")
@Data
public class Task {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;
    private String description;
    private LocalDateTime deadline;

    /** 任务状态：支持静默搁置、归档等 */
    @Enumerated(EnumType.STRING)
    @Column(name = "status", nullable = true)
    private TaskStatus status;

    /** 任务粒度：为模糊任务提示预留 */
    @Enumerated(EnumType.STRING)
    @Column(name = "granularity", nullable = true)
    private TaskGranularity granularity;

    /** 精力需求（HIGH/MEDIUM/LOW），用于弱时间管理推荐 */
```

```
    @Enumerated(EnumType.STRING)
    @Column(name = "energy_requirement", nullable = true)
    private EnergyLevel energyRequirement;

    /** 心理负担（HEAVY/LIGHT） */
    @Enumerated(EnumType.STRING)
    @Column(name = "mental_load", nullable = true)
    private MentalLoad mentalLoad;

    /** 偏好时间段（上午/下午/晚上） */
    @Enumerated(EnumType.STRING)
    @Column(name = "preferred_slot", nullable = true)
    private TimeSlot preferredSlot;

    /** 预估分钟数（ATOMIC/STANDARD 时可选） */
    @Column(name = "estimated_minutes", nullable = true)
    private Integer estimatedMinutes;

    /** 创建时间，用于「存在天数」提示 */
    @CreationTimestamp
    @Column(name = "created_at")
    private LocalDateTime createdAt;

    /** 搁置时间（status=SHELVED 时） */
    @Column(name = "shelved_at", nullable = true)
    private LocalDateTime shelvedAt;

    @ManyToOne
    @JoinColumn(name = "user_id")
    private User user;

    /** 获取有效状态，null 视为 PENDING（兼容旧数据） */
    public TaskStatus getEffectiveStatus() {
        return status != null ? status : TaskStatus.PENDING;
    }
}
// ===== File: src/main/java/org/zhzssp/memorandum/entity/TaskGranular
package org.zhzssp.memorandum.entity;

/**
 * 任务粒度：为后续「原子任务 / 标准任务 / 模糊任务」提示预留。
 */
public enum TaskGranularity {
    /** 原子任务（≤15min） */
    ATOMIC,
    /** 标准任务（30–90min） */
    STANDARD,
```

```
    /** 模糊任务（如"复习数据库"） */
    FUZZY
}
// ===== File: src/main/java/org/zhzssp/memorandum/entity/TaskStatus.j
package org.zhzssp.memorandum.entity;

/**
 * 任务状态枚举，支持「静默搁置」「自然过期」等非惩罚式设计。
 */
public enum TaskStatus {
    /** 待办 */
    PENDING,
    /** 已完成 */
    DONE,
    /** 已归档 */
    ARCHIVED,
    /** 已搁置（静默搁置，不显示在默认视图） */
    SHELVED
}
// ===== File: src/main/java/org/zhzssp/memorandum/entity/TimeSlot.jav
package org.zhzssp.memorandum.entity;

/**
 * 偏好时间段：上午 / 下午 / 晚上。
 */
public enum TimeSlot {
    MORNING,
    AFTERNOON,
    EVENING
}

// ===== File: src/main/java/org/zhzssp/memorandum/entity/User.java ==
package org.zhzssp.memorandum.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Column;
import jakarta.persistence.Table;
import lombok.Data;

// Spring 启动时，会扫描所有带 @Entity 的类，--> 被Spring注册并管理
// 并根据属性自动在数据库中生成对应的表结构 --> JDBC
@Entity
@Table(name="user")
@Data
```

```
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column (unique = true)
    private String username;
    private String password;
}

// ===== File: src/main/java/org/zhzssp/memorandum/entity/UserPreferen
package org.zhzssp.memorandum.entity;

import jakarta.persistence.*;
import lombok.Data;

/**
 * 用户偏好设置：认知负载控制。
 */
@Entity
@Table(name = "user_preference")
@Data
public class UserPreference {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @OneToOne
    @JoinColumn(name = "user_id", unique = true)
    private User user;

    /** 每屏最多显示任务数（null 表示不限制） */
    @Column(name = "max_visible_tasks", nullable = true)
    private Integer maxVisibleTasks;

    /** 是否显示未来任务 */
    @Column(name = "show_future_tasks", nullable = false)
    private Boolean showFutureTasks = true;

    /** 是否显示统计信息（完成率等） */
    @Column(name = "show_statistics", nullable = false)
    private Boolean showStatistics = false;

    /** 默认思维模式（用于选择页预选、切换模式后默认选项） */
    @Enumerated(EnumType.STRING)
    @Column(name = "default_mindset_mode", nullable = true)
    private MindsetMode defaultMindsetMode;
```

```
    /** 规划模式下的默认任务视图：today / slot / energy */
    @Column(name = "default_task_view", length = 20, nullable = true)
    private String defaultTaskView;

    /** 模糊任务存在超过 N 天则提示拆分；0 表示不提示 */
    @Column(name = "fuzzy_task_days_threshold", nullable = true)
    private Integer fuzzyTaskDaysThreshold;

    /** 规划模式下是否显示目标管理区块 */
    @Column(name = "show_goals", nullable = true)
    private Boolean showGoals;

    /** 是否显示「需要拆分的模糊任务」提示 */
    @Column(name = "show_fuzzy_hint", nullable = true)
    private Boolean showFuzzyHint;

    /** 是否显示归档/搁置任务区块 */
    @Column(name = "show_archived_section", nullable = true)
    private Boolean showArchivedSection;

    /** 是否显示规划完成得分曲线区块 */
    @Column(name = "show_score_section", nullable = true)
    private Boolean showScoreSection;

    /** 背景亮/暗：light / dark，默认亮色 */
    @Column(name = "theme", length = 10, nullable = true)
    private String theme;

    public enum MindsetMode {
        EXECUTE,  // 执行模式
        LEARN,    // 学习模式
        PLAN      // 规划模式
    }
}
// ===== File: src/main/java/org/zhzssp/memorandum/feature/goal/contro
package org.zhzssp.memorandum.feature.goal.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.*;
import org.zhzssp.memorandum.entity.GoalType;
import org.zhzssp.memorandum.entity.User;
import org.zhzssp.memorandum.feature.goal.entity.Goal;
import org.zhzssp.memorandum.feature.goal.service.GoalService;
import org.zhzssp.memorandum.repository.UserRepository;
```

```
import java.security.Principal;
import java.util.List;

/**
 * 目标控制器（插件层）。
 */
@Controller
@RequestMapping("/goal")
public class GoalController {

    @Autowired
    private GoalService goalService;

    @Autowired
    private UserRepository userRepository;

    @GetMapping("/list")
    @ResponseBody
    public List<Goal> listGoals(Principal principal) {
        User user = userRepository.findByUsername(principal.getName())
        return goalService.findActiveGoalsByUser(user);
    }

    @PostMapping("/add")
    @ResponseBody
    public Long addGoal(@RequestParam String name,
                        @RequestParam(required = false) String goalType
                        Principal principal) {
        User user = userRepository.findByUsername(principal.getName())
        Goal goal = new Goal();
        goal.setName(name);
        if (goalType != null && !goalType.isEmpty()) {
            try {
                goal.setGoalType(GoalType.valueOf(goalType));
            } catch (IllegalArgumentException ignored) {}
        }
        goal.setUser(user);
        return goalService.save(goal).getId();
    }

    @PostMapping("/archive/{id}")
    @ResponseBody
    public String archiveGoal(@PathVariable Long id, Principal princip
        try {
            User user = userRepository.findByUsername(principal.getNam
            goalService.archive(id, user);
            return "success";
```

```
        } catch (Exception e) {
            return "error:failed";
        }
    }

    /**
     * 删除目标：
     * mode = deleteTasks -> 级联删除绑定任务
     * mode = keepTasks   -> 仅删除目标和关联，保留任务
     */
    @PostMapping("/delete/{id}")
    @ResponseBody
    public String deleteGoal(@PathVariable Long id,
                             @RequestParam(name = "mode", defaultValue
                             Principal principal) {
        try {
            User user = userRepository.findByUsername(principal.getNam
            goalService.deleteGoalWithMode(id, user, mode);
            return "success";
        } catch (Exception e) {
            return "error:failed";
        }
    }
}
// ===== File: src/main/java/org/zhzssp/memorandum/feature/goal/dto/Go
package org.zhzssp.memorandum.feature.goal.dto;

import org.zhzssp.memorandum.entity.Task;
import org.zhzssp.memorandum.feature.goal.entity.Goal;

import java.util.List;

/** 用于树状展示：一个目标及其关联的任务（仅含有关联关系的） */
public record GoalWithTasks(Goal goal, List<Task> tasks) {}
// ===== File: src/main/java/org/zhzssp/memorandum/feature/goal/dto/Tr
package org.zhzssp.memorandum.feature.goal.dto;

import com.fasterxml.jackson.annotation.JsonInclude;

import java.util.List;

/** D3 树状图节点：目标与任务层级，用于 JSON API */
@JsonInclude(JsonInclude.Include.NON_NULL)
public record TreeNodeDto(
    String name,
    String type,
    Long id,
```

```
    String goalType,
    String deadline,
    List<TreeNodeDto> children
) {}
// ===== File: src/main/java/org/zhzssp/memorandum/feature/goal/entity
package org.zhzssp.memorandum.feature.goal.entity;

import jakarta.persistence.*;
import lombok.Data;
import org.hibernate.annotations.CreationTimestamp;
import org.zhzssp.memorandum.entity.GoalType;
import org.zhzssp.memorandum.entity.User;

import java.time.LocalDateTime;

/**
 * 目标实体（插件层）。弱关联设计：不强制树状，支持临时/模糊目标。
 */
@Entity
@Table(name = "goal")
@Data
public class Goal {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @Enumerated(EnumType.STRING)
    @Column(name = "goal_type", nullable = true)
    private GoalType goalType;

    @CreationTimestamp
    @Column(name = "created_at")
    private LocalDateTime createdAt;

    /** 归档时间（软删除，null 表示有效） */
    @Column(name = "archived_at", nullable = true)
    private LocalDateTime archivedAt;

    @ManyToOne
    @JoinColumn(name = "user_id")
    private User user;
}
// ===== File: src/main/java/org/zhzssp/memorandum/feature/goal/listen
package org.zhzssp.memorandum.feature.goal.listener;
```

```java
import org.springframework.context.event.EventListener;
import org.springframework.stereotype.Component;
import org.zhzssp.memorandum.core.event.TaskCompletedEvent;
import org.zhzssp.memorandum.core.event.TaskCreatedEvent;
import org.zhzssp.memorandum.entity.Link;
import org.zhzssp.memorandum.entity.TaskStatus;
import org.zhzssp.memorandum.feature.goal.entity.Goal;
import org.zhzssp.memorandum.feature.goal.repository.GoalRepository;
import org.zhzssp.memorandum.repository.LinkRepository;

import java.util.List;

/**
 * 目标功能的事件监听器（插件层）。
 * 通过监听核心事件来扩展目标相关行为，而不直接修改核心代码。
 */
@Component
public class GoalEventListener {

    private final GoalRepository goalRepository;
    private final LinkRepository linkRepository;

    public GoalEventListener(GoalRepository goalRepository, LinkReposi
        this.goalRepository = goalRepository;
        this.linkRepository = linkRepository;
    }

    /**
     * 监听任务创建事件。
     * 可以用于：目标聚类、任务分析等（未来扩展）。
     */
    @EventListener
    public void onTaskCreated(TaskCreatedEvent event) {
        // 未来可以在这里实现：自动聚类任务到目标、分析任务类型等
        // 目前保持空实现，展示事件驱动的扩展点
    }

    /**
     * 监听任务完成事件。
     * 可以用于：更新目标进度、推荐相关笔记等（未来扩展）。
     */
    @EventListener
    public void onTaskCompleted(TaskCompletedEvent event) {
        // 未来可以在这里实现：更新目标完成度、推荐学习笔记等
        // 目前保持空实现，展示事件驱动的扩展点
    }
```

```
}
// ===== File: src/main/java/org/zhzssp/memorandum/feature/goal/reposi
package org.zhzssp.memorandum.feature.goal.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.zhzssp.memorandum.entity.User;
import org.zhzssp.memorandum.feature.goal.entity.Goal;

import java.util.List;

public interface GoalRepository extends JpaRepository<Goal, Long> {

    List<Goal> findByUser(User user);

    /** 有效目标（未归档） */
    List<Goal> findByUserAndArchivedAtIsNull(User user);
}
// ===== File: src/main/java/org/zhzssp/memorandum/feature/goal/servic
package org.zhzssp.memorandum.feature.goal.service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.zhzssp.memorandum.entity.Link;
import org.zhzssp.memorandum.entity.Task;
import org.zhzssp.memorandum.entity.TaskStatus;
import org.zhzssp.memorandum.entity.User;
import org.zhzssp.memorandum.feature.goal.dto.GoalWithTasks;
import org.zhzssp.memorandum.feature.goal.entity.Goal;
import org.zhzssp.memorandum.feature.goal.repository.GoalRepository;
import org.zhzssp.memorandum.repository.LinkRepository;
import org.zhzssp.memorandum.repository.TaskRepository;

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;
import java.util.Set;
import java.util.stream.Collectors;

/**
 * 目标服务（插件层）。Task↔Goal 多对多弱关联通过 Link 表实现。
 */
@Service
public class GoalService {

    @Autowired
    private GoalRepository goalRepository;
```

```
@Autowired
private LinkRepository linkRepository;

@Autowired
private TaskRepository taskRepository;

public List<Goal> findActiveGoalsByUser(User user) {
    return goalRepository.findByUserAndArchivedAtIsNull(user);
}

/**
 * 获取「仅含有关联关系」的目标与任务树：每个目标下列出与其关联的任务。
 * 无关联的目标或任务不会出现在结果中。
 */
public List<GoalWithTasks> findGoalTaskTree(User user) {
    List<Goal> activeGoals = goalRepository.findByUserAndArchivedA
    List<GoalWithTasks> result = new ArrayList<>();
    for (Goal goal : activeGoals) {
        List<Link> links = linkRepository.findByTargetTypeAndTarge
        List<Long> taskIds = links.stream()
                .filter(l -> l.getSourceType() == Link.LinkSourceT
                .map(Link::getSourceId)
                .distinct()
                .toList();
        if (taskIds.isEmpty()) continue;
        List<Task> tasks = taskRepository.findAllById(taskIds).str
                .filter(t -> t.getUser() != null && t.getUser().ge
                .toList();
        if (tasks.isEmpty()) continue;
        result.add(new GoalWithTasks(goal, tasks));
    }
    return result;
}

public List<Goal> findGoalsByUser(User user) {
    return goalRepository.findByUser(user);
}

/** 获取任务关联的目标 ID 列表 */
public List<Long> findGoalIdsByTaskId(Long taskId) {
    return linkRepository.findBySourceTypeAndSourceId(Link.LinkSou
            .stream()
            .filter(l -> l.getTargetType() == Link.LinkTargetType.
            .map(Link::getTargetId)
            .collect(Collectors.toList());
}
```

```
    /** 获取任务关联的目标（需校验 goal 属于当前用户） */
    public List<Goal> findGoalsByTaskId(Long taskId, User user) {
        List<Long> goalIds = findGoalIdsByTaskId(taskId);
        if (goalIds.isEmpty()) return List.of();
        return goalRepository.findAllById(goalIds).stream()
                .filter(g -> g.getUser().getId().equals(user.getId()))
                .collect(Collectors.toList());
    }

    /** 绑定任务到目标（替换原有绑定） */
    public void linkTaskToGoals(Long taskId, List<Long> goalIds, User
        // 删除原有 TASK->GOAL 链接
        List<Link> existing = linkRepository.findBySourceTypeAndSource
        existing.stream()
                .filter(l -> l.getTargetType() == Link.LinkTargetType.
                .forEach(linkRepository::delete);

        // 校验 goal 归属后创建新链接
        if (goalIds != null && !goalIds.isEmpty()) {
            List<Goal> goals = goalRepository.findAllById(goalIds);
            for (Goal g : goals) {
                if (g.getUser().getId().equals(user.getId())) {
                    Link link = new Link();
                    link.setSourceType(Link.LinkSourceType.TASK);
                    link.setSourceId(taskId);
                    link.setTargetType(Link.LinkTargetType.GOAL);
                    link.setTargetId(g.getId());
```

后 30 页（约 1800 行）

```
.theme-dark .goal-delete-subtext {
    color: #c9d1d9;
}

.theme-dark .status-tag {
    color: #8b949e;
}

.theme-dark .goal-task-tree-section {
    background: #252d3a;
    border-color: #30363d;
}
```

```
.theme-dark .goal-task-tree-section h3 {
    color: #e6edf3;
}

.theme-dark .goal-tree-desc,
.theme-dark .goal-tree-empty {
    color: #8b949e;
}

.theme-dark .goal-tree-link {
    stroke: #484f58;
}

.theme-dark .goal-tree-label {
    fill: #e6edf3;
}

.theme-dark .goal-tree-node-goal circle {
    fill: #58a6ff;
    stroke: #21262d;
}

.theme-dark .goal-tree-node-task circle {
    fill: #3fb950;
    stroke: #21262d;
}
// ===== File: src/main/resources/static/css/filter.css =====
/* Filter/Search bar styles */
.filter-wrapper {
    margin-bottom: 30px;
    display: flex;
    flex-wrap: wrap;
    gap: 12px;
    align-items: center;
}

.filter-wrapper input[type="text"],
.filter-wrapper input[type="date"] {
    padding: 10px 14px;
    border: 1px solid #cfd9e4;
    border-radius: 8px;
    font-size: 14px;
    outline: none;
    transition: box-shadow 0.2s ease, border-color 0.2s ease;
}

.filter-wrapper input[type="text"]:focus,
```

```
.filter-wrapper input[type="date"]:focus {
    border-color: #4a90e2;
    box-shadow: 0 0 0 3px rgba(74, 144, 226, 0.2);
}

.filter-btn {
    background: #4a90e2;
    color: #fff;
    border: none;
    border-radius: 8px;
    padding: 10px 20px;
    font-weight: 600;
    font-size: 14px;
    cursor: pointer;
    transition: background 0.2s ease, transform 0.2s ease;
}

.filter-btn:hover {
    background: #357abd;
    transform: translateY(-2px);
}

@media (max-width: 540px) {
    .filter-wrapper {
        flex-direction: column;
        align-items: stretch;
    }

    .filter-btn {
        width: 100%;
    }
}
// ===== File: src/main/resources/static/css/login.css =====
/* 页面背景和字体 */
body {
    display: flex;
    justify-content: center;
    align-items: center;
    height: 100vh;
    margin: 0;
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    background: linear-gradient(135deg, #6a11cb, #2575fc);
}

/* 登录容器 */
.login-container {
    background-color: #fff;
```

```
    padding: 50px 40px;
    border-radius: 16px;
    box-shadow: 0 15px 35px rgba(0,0,0,0.2);
    width: 380px;
    text-align: center;
    position: relative;
    animation: fadeIn 0.8s ease;
}

/* 标题 */
.login-container h2 {
    margin-bottom: 30px;
    color: #333;
    font-size: 28px;
    font-weight: 600;
}

/* 输入框 */
.login-container input {
    width: 100%;
    padding: 14px 15px;
    margin: 12px 0;
    border-radius: 10px;
    border: 1px solid #ccc;
    font-size: 16px;
    transition: all 0.3s ease;
}

.login-container input:focus {
    border-color: #2575fc;
    box-shadow: 0 0 8px rgba(37,117,252,0.3);
    outline: none;
}

/* 按钮 */
.login-container button {
    width: 100%;
    padding: 14px;
    margin-top: 12px;
    border: none;
    border-radius: 10px;
    font-size: 16px;
    cursor: pointer;
    transition: 0.3s;
}

/* 登录按钮 */
```

```
.login-btn {
    background-color: #2575fc;
    color: #fff;
    font-weight: 500;
}

.login-btn:hover {
    background-color: #0056d2;
}

/* 注册按钮 */
.register-btn {
    background-color: #6c757d;
    color: #fff;
    font-weight: 500;
}

.register-btn:hover {
    background-color: #5a6268;
}

/* 表单布局 */
.login-container form {
    display: flex;
    flex-direction: column;
}

/* 错误提示 */
.error-message {
    margin-top: 15px;
    color: #d93025;
    font-weight: 500;
    font-size: 14px;
    text-align: left;
}

/* 动画效果 */
@keyframes fadeIn {
    from {opacity: 0; transform: translateY(-20px);}
    to {opacity: 1; transform: translateY(0);}
}
// ===== File: src/main/resources/static/css/register.css =====
/* 页面背景 */
body {
    display: flex;
    justify-content: center;
    align-items: center;
```

```
    height: 100vh;
    margin: 0;
    font-family: 'Arial', sans-serif;
    background: linear-gradient(120deg, #ff758c, #ff7eb3);
    color: #333;
}

/* 注册容器 */
.register-container {
    background-color: #ffffffdd; /* 半透明白色 */
    padding: 40px 35px;
    border-radius: 15px;
    box-shadow: 0 12px 30px rgba(0, 0, 0, 0.25);
    width: 360px;
    text-align: center;
}

/* 标题 */
.register-container h2 {
    margin-bottom: 25px;
    font-size: 28px;
    color: #ff4e6d;
}

/* 输入框样式 */
.register-container input {
    width: 100%;
    padding: 12px 15px;
    margin: 10px 0;
    border-radius: 10px;
    border: 1px solid #ff7eb3;
    font-size: 16px;
}

/* 按钮样式 */
.register-container button {
    width: 100%;
    padding: 12px;
    margin-top: 10px;
    border: none;
    border-radius: 10px;
    font-size: 16px;
    cursor: pointer;
    transition: 0.3s;
}

/* 注册按钮 */
```

```
.register-container button.register-btn {
    background-color: #ff4e6d;
    color: white;
}

.register-container button.register-btn:hover {
    background-color: #e13e5a;
}

/* 返回登录按钮 */
.register-container button.login-btn {
    background-color: #f0f0f0;
    color: #333;
}

.register-container button.login-btn:hover {
    background-color: #dcdcdc;
}

/* 表单布局 */
.register-container form {
    display: flex;
    flex-direction: column;
}
// ===== File: src/main/resources/static/css/selectFeatures.css =====
body {
    font-family: "Segoe UI", Arial, sans-serif;
    background: linear-gradient(135deg, #f0f4ff, #dfe9f3);
    margin: 0;
    padding: 0;
}

.container {
    max-width: 500px;
    margin: 80px auto;
    background: white;
    border-radius: 16px;
    box-shadow: 0 8px 25px rgba(0, 0, 0, 0.1);
    padding: 40px 60px;
    text-align: center;
}

h2 {
    color: #2c3e50;
    margin-bottom: 30px;
}
```

```
.feature-option {
    display: block;
    margin-bottom: 20px;
    padding: 16px 20px;
    border: 2px solid #e1e7f0;
    border-radius: 12px;
    cursor: pointer;
    transition: all 0.25s ease;
    text-align: left;
}

.feature-option:hover {
    border-color: #4a90e2;
    background: #f0f7ff;
    transform: translateY(-2px);
    box-shadow: 0 4px 12px rgba(0, 0, 0, 0.08);
}

.feature-option input {
    margin-right: 12px;
    transform: scale(1.2);
    cursor: pointer;
}

.feature-option label {
    cursor: pointer;
    display: block;
}

.feature-option strong {
    display: block;
    font-size: 1.1rem;
    color: #2c3e50;
    margin-bottom: 4px;
}

.feature-option small {
    display: block;
    font-size: 0.85em;
    color: #7f8c8d;
    line-height: 1.4;
}

.submit-btn {
    display: inline-block;
    background: #4a90e2;
    color: #fff;
```

```
    border: none;
    border-radius: 10px;
    padding: 12px 26px;
    font-size: 1rem;
    font-weight: bold;
    cursor: pointer;
    transition: background 0.25s ease, transform 0.25s ease;
}

.submit-btn:hover {
    background: #357abd;
    transform: scale(1.05);
}
// ===== File: src/main/resources/static/js/dashboard-goals.js =====
document.addEventListener('DOMContentLoaded', () => {
    const goalsSection = document.getElementById('goalsSection');
    if (!goalsSection) return;

    const modal = document.getElementById('goalModal');
    const openBtn = document.getElementById('openGoalModal');
    const closeBtn = document.getElementById('closeGoalModal');
    const saveBtn = document.getElementById('saveGoalBtn');
    const goalNameInput = document.getElementById('goalName');
    const goalTypeSelect = document.getElementById('goalType');
    const goalList = document.getElementById('goalList');

    // 删除确认弹窗
    const deleteModal = document.getElementById('goalDeleteModal');
    const keepTasksBtn = document.getElementById('keepTasksBtn');
    const deleteTasksBtn = document.getElementById('deleteTasksBtn');
    const cancelDeleteBtn = document.getElementById('cancelDeleteGoalB
    let currentDeleteGoalId = null;

    const csrfToken = document.querySelector('meta[name="_csrf"]')?.ge
    const csrfHeader = document.querySelector('meta[name="_csrf_header

    function archiveGoal(goalId) {
        const body = new URLSearchParams({ _csrf: csrfToken });
        fetch(`/goal/archive/${goalId}`, {
            method: 'POST',
            headers: {
                'Content-Type': 'application/x-www-form-urlencoded',
                [csrfHeader]: csrfToken
            },
            body: body
        }).then(r => {
            if (r.ok) location.reload();
```

```
    });
}

function deleteGoal(goalId) {
    currentDeleteGoalId = goalId;
    if (deleteModal) {
        deleteModal.style.display = 'flex';
    }
}

openBtn?.addEventListener('click', () => {
    goalNameInput.value = '';
    goalTypeSelect.value = '';
    modal.style.display = 'flex';
});

closeBtn?.addEventListener('click', () => {
    modal.style.display = 'none';
});

saveBtn?.addEventListener('click', () => {
    const name = goalNameInput.value?.trim();
    if (!name) return;
    const goalType = goalTypeSelect.value || '';
    const formData = new URLSearchParams();
    formData.append('name', name);
    if (goalType) formData.append('goalType', goalType);
    formData.append('_csrf', csrfToken);

    fetch('/goal/add', {
        method: 'POST',
        headers: {
            'Content-Type': 'application/x-www-form-urlencoded',
            [csrfHeader]: csrfToken
        },
        body: formData
    }).then(r => {
        if (r.ok) {
            modal.style.display = 'none';
            location.reload();
        }
    });
});

goalList?.querySelectorAll('.goal-archive-btn').forEach(btn => {
    btn.addEventListener('click', () => archiveGoal(btn.dataset.go
});
```

```
    goalList?.querySelectorAll('.goal-delete-btn').forEach(btn => {
        btn.addEventListener('click', () => deleteGoal(btn.dataset.goa
    });

    // 删除确认弹窗事件绑定
    keepTasksBtn?.addEventListener('click', () => {
        if (!currentDeleteGoalId) return;
        const body = new URLSearchParams({ _csrf: csrfToken, mode: 'ke
        fetch(`/goal/delete/${currentDeleteGoalId}`, {
            method: 'POST',
            headers: {
                'Content-Type': 'application/x-www-form-urlencoded',
                [csrfHeader]: csrfToken
            },
            body
        }).then(r => {
            if (r.ok) location.reload();
        });
    });

    deleteTasksBtn?.addEventListener('click', () => {
        if (!currentDeleteGoalId) return;
        const body = new URLSearchParams({ _csrf: csrfToken, mode: 'de
        fetch(`/goal/delete/${currentDeleteGoalId}`, {
            method: 'POST',
            headers: {
                'Content-Type': 'application/x-www-form-urlencoded',
                [csrfHeader]: csrfToken
            },
            body
        }).then(r => {
            if (r.ok) location.reload();
        });
    });

    const closeDeleteModal = () => {
        if (deleteModal) deleteModal.style.display = 'none';
        currentDeleteGoalId = null;
    };

    cancelDeleteBtn?.addEventListener('click', closeDeleteModal);
    // 点击遮罩关闭
    deleteModal?.addEventListener('click', (e) => {
        if (e.target === deleteModal) closeDeleteModal();
    });
});
```

```
// ===== File: src/main/resources/static/js/dashboard-notes.js =====
document.addEventListener('DOMContentLoaded', () => {

    const noteList = document.getElementById('noteList');
    // 笔记模式下方渲染 notesSection，任务模式下不运行
    if (!noteList) return;

    const modal = document.getElementById('noteModal');
    const openBtn = document.getElementById('openNoteModal');
    const closeBtn = document.getElementById('closeNoteModal');
    const saveBtn = document.getElementById('saveNoteBtn');
    const titleInput = document.getElementById('noteTitle');

    const csrfToken = document.querySelector('meta[name="_csrf"]')?.ge
    const csrfHeader = document.querySelector('meta[name="_csrf_header

    // 类型切换
    let currentNoteType = 'SCRATCH';
    const typeButtons = document.querySelectorAll('.note-type-btn');

    typeButtons.forEach(btn => {
        btn.addEventListener('click', () => {
            currentNoteType = btn.dataset.type;
            typeButtons.forEach(b => b.classList.toggle('active', b ==
            document.querySelectorAll('.note-panel').forEach(p => p.st
            const panel = document.getElementById(`notePanel-${current
            if (panel) panel.style.display = 'block';
        });
    });

    function fetchNotes() {
        fetch('/note/list')
            .then(r => r.json())
            .then(data => {
                noteList.innerHTML = '';
                data.forEach(n => {
                    const li = document.createElement('li');
                    const typeLabel = n.type || 'SCRATCH';
                    li.textContent = `[${typeLabel}] ${n.title || '(无
                    noteList.appendChild(li);
                });
            });
    }

    openBtn?.addEventListener('click', () => {
        modal.style.display = 'flex';
    });
```

```
closeBtn?.addEventListener('click', () => {
    modal.style.display = 'none';
});

saveBtn?.addEventListener('click', () => {
    let finalTitle = titleInput.value.trim();
    let content = '';

    if (currentNoteType === 'SCRATCH') {
        const c = document.getElementById('scratchContent').value.
        content = c;
        if (!finalTitle && c) finalTitle = c.slice(0, 20);
    } else if (currentNoteType === 'LEARNING') {
        const concept    = document.getElementById('learningConcep
        const definition = document.getElementById('learningDefini
        const example    = document.getElementById('learningExampl
        finalTitle = finalTitle || concept || '学习笔记';
        content =
            (concept    ? `# 概念\n${concept}\n\n` : '') +
            (definition ? `# 定义 / 要点\n${definition}\n\n` : '')
            (example    ? `# 示例\n${example}\n` : '');
    } else if (currentNoteType === 'PROJECT') {
        const ctx    = document.getElementById('projectContext').v
        const todos  = document.getElementById('projectTodos').val
        const issues = document.getElementById('projectIssues').va
        finalTitle = finalTitle || '项目笔记';
        content =
            (ctx    ? `# 背景 / 决策\n${ctx}\n\n` : '') +
            (todos  ? `# TODO\n${todos}\n\n` : '') +
            (issues ? `# 问题 / 风险\n${issues}\n` : '');
    } else if (currentNoteType === 'RETROSPECTIVE') {
        const what    = document.getElementById('retroWhat').value
        const why     = document.getElementById('retroWhy').value.
        const lessons = document.getElementById('retroLessons').va
        finalTitle = finalTitle || '复盘笔记';
        content =
            (what    ? `# 发生了什么\n${what}\n\n` : '') +
            (why     ? `# 原因分析\n${why}\n\n` : '') +
            (lessons ? `# 教训 / 改进\n${lessons}\n` : '');
    }

    const body = { title: finalTitle, content, type: currentNoteTy

    fetch('/note/add', {
        method: 'POST',
        headers: {
```

```
                'Content-Type': 'application/json',
                [csrfHeader]: csrfToken
            },
            body: JSON.stringify(body)
        }).then(r => {
            if (r.ok) {
                titleInput.value = '';
                modal.style.display = 'none';
                fetchNotes();
            }
        });
    });

    fetchNotes();
});
// ===== File: src/main/resources/static/js/dashboard.js =====
document.addEventListener('DOMContentLoaded', function () {
    const contextMenu = document.getElementById('contextMenu');
    const memoItems = document.querySelectorAll('.memo-item');
    let currentMemoId = null;

    // 为每个memo项添加右键事件监听器
    memoItems.forEach(item => {
        item.addEventListener('contextmenu', function (e) {
            e.preventDefault();
            currentMemoId = this.getAttribute('data-memo-id');

            // 显示右键菜单
            contextMenu.style.display = 'block';
            contextMenu.style.left = e.pageX + 'px';
            contextMenu.style.top = e.pageY + 'px';
        });
    });

    // 完成按钮事件
    document.getElementById('completeBtn').addEventListener('click', f
        if (currentMemoId) {
            updateTaskStatus(currentMemoId, 'complete');
        }
        hideContextMenu();
    });

    // 搁置按钮事件
    document.getElementById('shelveBtn').addEventListener('click', fun
        if (currentMemoId) {
            updateTaskStatus(currentMemoId, 'shelve');
        }
```

```
    hideContextMenu();
});

// 删除按钮事件
document.getElementById('deleteBtn').addEventListener('click', fun
    if (currentMemoId) {
        deleteMemo(currentMemoId);
    }
    hideContextMenu();
});

// 取消按钮事件
document.getElementById('cancelBtn').addEventListener('click', fun
    hideContextMenu();
});

// 点击其他地方隐藏菜单
document.addEventListener('click', function (e) {
    if (!contextMenu.contains(e.target)) {
        hideContextMenu();
    }
});

// 隐藏右键菜单
function hideContextMenu() {
    contextMenu.style.display = 'none';
    currentMemoId = null;
}

// 更新任务状态（完成/搁置）
function updateTaskStatus(taskId, action) {
    const csrfToken = document.querySelector('meta[name="_csrf"]')
        document.querySelector('input[name="_csrf"]')?.value;
    const url = action === 'complete' ? `/memo/complete/${taskId}`

    fetch(url, {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json',
            'X-CSRF-TOKEN': csrfToken
        }
    })
        .then(response => response.ok ? response.text() : Promise.
        .then(data => {
            if (data === 'success') {
                location.reload();
            } else {
```

```
                alert('操作失败: ' + data);
            }
        })
        .catch(err => {
            console.error('Error:', err);
            alert('操作失败: ' + err.message);
        });
}

// 删除任务
function deleteMemo(memoId) {
    // 获取CSRF token
    const csrfToken = document.querySelector('meta[name="_csrf"]')
        document.querySelector('input[name="_csrf"]')?.value;

    fetch(`/memo/delete/${memoId}`, {
        method: 'DELETE',
        headers: {
            'Content-Type': 'application/json',
            'X-CSRF-TOKEN': csrfToken
        }
    })
        .then(response => {
            if (response.ok) {
                return response.text();
            } else {
                throw new Error('HTTP ' + response.status + ': ' +
            }
        })
        .then(data => {
            if (data === 'success') {
                // 删除成功后重新加载页面
                location.reload();
            } else {
                alert('删除失败: ' + data);
            }
        })
        .catch(error => {
            console.error('Error:', error);
            alert('删除失败: ' + error.message);
        });
}

// 目标与任务树：右上角按键控制显示/隐藏（默认隐藏），首次显示时拉取数
const toggleGoalTreeBtn = document.getElementById('toggleGoalTreeB
const goalTaskTreeSection = document.getElementById('goalTaskTreeS
if (toggleGoalTreeBtn && goalTaskTreeSection) {
```

```
        toggleGoalTreeBtn.addEventListener('click', function () {
            const hidden = goalTaskTreeSection.style.display === 'none
            goalTaskTreeSection.style.display = hidden ? 'block' : 'no
            goalTaskTreeSection.setAttribute('aria-hidden', hidden ? '
            if (hidden && typeof window.initGoalTreeViz === 'function'
                window.initGoalTreeViz();
            }
        });
    }
});
// ===== File: src/main/resources/static/js/goal-tree-viz.js =====
/**
 * 目标与任务树状图：使用 D3 绘制节点与边，直观展示关联关系。
 * 在展示区块首次显示时拉取 /api/goal-task-tree 并渲染。
 */
(function () {
    var rendered = false;
    var svgWidth = 900;
    var svgHeight = 420;

    function getCsrfHeaders() {
        var token = document.querySelector('meta[name="_csrf"]');
        var header = document.querySelector('meta[name="_csrf_header"]
        if (!token || !header) return {};
        var h = {};
        h[header.getAttribute('content')] = token.getAttribute('conten
        return h;
    }

    function drawTree(data) {
        var container = document.getElementById('goalTreeVizContainer'
        var emptyMsg = document.getElementById('goalTreeEmptyMsg');
        var svgEl = document.getElementById('goalTreeSvg');
        if (!container || !svgEl) return;

        if (!data || !data.children || data.children.length === 0) {
            emptyMsg.style.display = 'block';
            svgEl.style.display = 'none';
            return;
        }
        emptyMsg.style.display = 'none';
        svgEl.style.display = 'block';

        // 清空旧图
        svgEl.innerHTML = '';
        var svg = d3.select(svgEl)
            .attr('width', '100%')
```

```
    .attr('height', svgHeight)
    .attr('viewBox', [0, 0, svgWidth, svgHeight]);

var g = svg.append('g');

var root = d3.hierarchy(data);
var treeLayout = d3.tree().size([svgWidth - 80, svgHeight - 60
treeLayout(root);

// 整体留边距（d3.tree 中 x=水平, y=深度）
var marginLeft = 50;
var marginTop = 36;
root.each(function (d) {
    d.x += marginLeft;
    d.y += marginTop;
});

// 边：从父到子（自上而下：x 水平，y 垂直）
var linkGen = d3.linkVertical()
    .x(function (d) { return d.x; })
    .y(function (d) { return d.y; });

g.selectAll('.goal-tree-link')
    .data(root.links())
    .enter()
    .append('path')
    .attr('class', 'goal-tree-link')
    .attr('d', linkGen)
    .attr('fill', 'none')
    .attr('stroke', '#c0c8d4')
    .attr('stroke-width', 1.5);

// 节点组（圆 + 文字）
var node = g.selectAll('.goal-tree-node')
    .data(root.descendants())
    .enter()
    .append('g')
    .attr('class', function (d) {
        return 'goal-tree-node goal-tree-node-' + (d.data.type
    })
    .attr('transform', function (d) { return 'translate(' + d.

// 根节点不画或画小点
node.filter(function (d) { return d.data.type === 'root'; })
    .append('circle')
    .attr('r', 0)
    .attr('fill', 'transparent');
```

```
        node.filter(function (d) { return d.data.type !== 'root'; })
            .append('circle')
            .attr('r', function (d) { return d.data.type === 'goal' ?
            .attr('fill', function (d) { return d.data.type === 'goal'
            .attr('stroke', '#fff')
            .attr('stroke-width', 2);

        // 标签
        node.filter(function (d) { return d.data.type !== 'root'; })
            .append('text')
            .attr('dy', function (d) { return d.data.type === 'goal' ?
            .attr('text-anchor', 'middle')
            .attr('class', 'goal-tree-label')
            .text(function (d) {
                var name = d.data.name || '';
                if (d.data.deadline) name += ' (' + d.data.deadline +
                return name.length > 24 ? name.slice(0, 22) + '…' : na
            });
    }

    window.initGoalTreeViz = function () {
        if (rendered) return;
        rendered = true;

        fetch('/api/goal-task-tree', { headers: getCsrfHeaders(), cred
            .then(function (r) { return r.json(); })
            .then(drawTree)
            .catch(function (err) {
                console.error('Goal tree load failed:', err);
                var emptyMsg = document.getElementById('goalTreeEmptyM
                var svgEl = document.getElementById('goalTreeSvg');
                if (emptyMsg) {
                    emptyMsg.textContent = '加载失败，请刷新重试。';
                    emptyMsg.style.display = 'block';
                }
                if (svgEl) svgEl.style.display = 'none';
            });
    };
})();
// ===== File: src/main/resources/static/js/insight-score.js =====
document.addEventListener('DOMContentLoaded', function () {
    const chartCanvas = document.getElementById('scoreChart');
    if (!chartCanvas) {
        return; // 仅在规划模式下存在
    }
```

```
const startInput = document.getElementById('scoreStart');
const endInput = document.getElementById('scoreEnd');
const refreshBtn = document.getElementById('scoreRefreshBtn');
const summaryBtn = document.getElementById('scoreSummaryBtn');
const summaryBox = document.getElementById('scoreSummary');

let scoreChart = null;

function formatDate(d) {
    const y = d.getFullYear();
    const m = String(d.getMonth() + 1).padStart(2, '0');
    const day = String(d.getDate()).padStart(2, '0');
    return `${y}-${m}-${day}`;
}

// 默认展示近 14 天
const today = new Date();
const start = new Date();
start.setDate(today.getDate() - 13);

if (!startInput.value) {
    startInput.value = formatDate(start);
}
if (!endInput.value) {
    endInput.value = formatDate(today);
}

function getCsrfToken() {
    const meta = document.querySelector('meta[name=\"_csrf\"]');
    if (meta) return meta.getAttribute('content');
    const input = document.querySelector('input[name=\"_csrf\"]');
    return input ? input.value : null;
}

function loadScores() {
    const startVal = startInput.value;
    const endVal = endInput.value;
    if (!startVal || !endVal) return;

    const csrfToken = getCsrfToken();

    fetch(`/insight/score?start=${startVal}&end=${endVal}`, {
        method: 'GET',
        headers: csrfToken ? { 'X-CSRF-TOKEN': csrfToken } : {}
    })
        .then(resp => resp.ok ? resp.json() : Promise.reject(new E
        .then(data => {
```

```
            const labels = data.map(d => d.date);
            const scores = data.map(d => d.totalScore);

            if (scoreChart) {
                scoreChart.destroy();
            }

            scoreChart = new Chart(chartCanvas.getContext('2d'), {
                type: 'line',
                data: {
                    labels: labels,
                    datasets: [{
                        label: '每日规划得分',
                        data: scores,
                        fill: false,
                        borderColor: '#4a90e2',
                        tension: 0.2,
                        pointRadius: 3
                    }]
                },
                options: {
                    responsive: true,
                    scales: {
                        y: {
                            beginAtZero: true,
                            max: 100
                        }
                    }
                }
            });
        })
        .catch(err => {
            console.error('加载得分数据失败', err);
        });
}

function loadSummary() {
    const startVal = startInput.value;
    const endVal = endInput.value;
    if (!startVal || !endVal) return;

    const csrfToken = getCsrfToken();

    if (summaryBox) {
        summaryBox.textContent = 'AI 正在分析这一段时间的规划完成情况
    }
```

```
        fetch(`/insight/score/summary?start=${startVal}&end=${endVal}`
            method: 'GET',
            headers: csrfToken ? { 'X-CSRF-TOKEN': csrfToken } : {}
        })
            .then(resp => resp.ok ? resp.json() : Promise.reject(new E
            .then(data => {
                if (summaryBox) {
                    summaryBox.textContent = data && data.summary
                        ? data.summary
                        : '没有获得有效的 AI 总结。';
                }
            })
            .catch(err => {
                console.error('加载 AI 总结失败', err);
                if (summaryBox) {
                    summaryBox.textContent = 'AI 总结暂时不可用，请稍后
                }
            });
    }

    if (refreshBtn) {
        refreshBtn.addEventListener('click', function () {
            loadScores();
        });
    }

    if (summaryBtn) {
        summaryBtn.addEventListener('click', function () {
            loadSummary();
        });
    }

    loadScores();
});

// ===== File: src/main/resources/templates/addMemo.html =====
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Add Memo</title>
    <link rel="stylesheet" th:href="@{/css/addMemo.css}">
</head>
<body>
<div class="memo-form-container">
    <h2>添加任务</h2>
    <form action="/memo/add" method="post">
```

```
<input type="text" name="title" placeholder="任务标题" require
<textarea name="description" placeholder="描述"></textarea>
<input type="datetime-local" name="deadline" required>

<div class="granularity-row">
    <label for="granularity">任务粒度（可选）</label>
    <select id="granularity" name="granularity">
        <option value="">不指定</option>
        <option value="ATOMIC">原子任务（≤15 分钟）</option>
        <option value="STANDARD">标准任务（30–90 分钟）</option
        <option value="FUZZY">模糊任务（待拆分）</option>
    </select>
</div>
<div class="estimate-row">
    <label for="estimatedMinutes">预估时长（分钟，可选）</label
    <input type="number" id="estimatedMinutes" name="estimated
</div>

<div class="time-layer-row">
    <label for="energyRequirement">精力需求（可选）</label>
    <select id="energyRequirement" name="energyRequirement">
        <option value="">不指定</option>
        <option value="HIGH">高精力</option>
        <option value="MEDIUM">中等</option>
        <option value="LOW">低精力</option>
    </select>
</div>
<div class="time-layer-row">
    <label for="mentalLoad">心理负担（可选）</label>
    <select id="mentalLoad" name="mentalLoad">
        <option value="">不指定</option>
        <option value="HEAVY">心理负担较重</option>
        <option value="LIGHT">心理负担较轻</option>
    </select>
</div>
<div class="time-layer-row">
    <label for="preferredSlot">偏好时间段（可选）</label>
    <select id="preferredSlot" name="preferredSlot">
        <option value="">不指定</option>
        <option value="MORNING">上午</option>
        <option value="AFTERNOON">下午</option>
        <option value="EVENING">晚上</option>
    </select>
</div>
<div class="goal-binding" th:if="${!goals.isEmpty()}">
    <label>绑定目标（可选，可多选）</label>
    <div class="goal-checkboxes">
```

```
                <label th:each="g : ${goals}">
                    <input type="checkbox" name="goalIds" th:value="${
                    <span th:text="${g.name}"></span>
                </label>
            </div>
        </div>
        <input type="hidden" th:name="${_csrf.parameterName}" th:value
        <button type="submit">添加</button>
    </form>
</div>
</body>
</html>
// ===== File: src/main/resources/templates/dashboard.html =====
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org" lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="_csrf" th:content="${_csrf.token}">
    <meta name="_csrf_header" th:content="${_csrf.headerName}">
    <title>Your Dashboard</title>
    <link rel="stylesheet" th:href="@{/css/dashboard.css}">
    <link rel="stylesheet" th:href="@{/css/filter.css}">
    <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
    <script src="https://cdn.jsdelivr.net/npm/d3@7"></script>
    <script th:src="@{/js/goal-tree-viz.js}"></script>
    <script th:src="@{/js/dashboard.js}"></script>
    <script th:src="@{/js/dashboard-notes.js}"></script>
    <script th:src="@{/js/dashboard-goals.js}"></script>
    <script th:src="@{/js/insight-score.js}"></script>
</head>

<body th:classappend="${theme == 'dark'} ? 'theme-dark' : ''">
<div class="container">
    <div class="top-bar">
        <span class="current-mode" th:text="${mindsetMode=='execute' ?
        <div>
            <button type="button" id="toggleGoalTreeBtn" class="switch
            <a th:href="@{/preference/settings(mode=${mindsetMode})}"
            <a href="/change-mode" class="switch-mode-btn">切换思维模式
        </div>
    </div>

    <!-- 目标与任务关系树：节点与边的树状图（默认隐藏，所有模式均可查看）
    <section id="goalTaskTreeSection" class="goal-task-tree-section" s
        <h3>目标与任务关系</h3>
        <p class="goal-tree-desc">仅展示有关联关系的目标及其下属任务，节
        <div id="goalTreeVizContainer" class="goal-tree-viz-container"
```

```
        <p id="goalTreeEmptyMsg" class="goal-tree-empty" style="di
        <svg id="goalTreeSvg" class="goal-tree-svg" aria-hidden="t
    </div>
</section>

<h2>Dashboard</h2>

<!-- 执行模式：低信息密度，只看今日+当前任务 -->
<div th:if="${mindsetMode=='execute'}">
    <h3>今日可行动任务</h3>
    <ul>
        <li th:each="memo : ${memos}" class="memo-item" th:data-me
            <div>
                <span class="title" th:text="${memo.title}"></span
                <br/>
                <small th:text="${memo.description}"></small>
            </div>
            <span class="deadline" th:text="${memo.deadline}"></sp
        </li>
    </ul>
    <a href="/memo/add" class="add-btn">＋ 添加任务</a>
</div>

<!-- 学习模式：中信息密度，任务+笔记并排 -->
<div th:if="${mindsetMode=='learn'}" class="learn-mode-layout">
    <div class="learn-tasks-column">
        <h3>今日任务</h3>
        <ul>
            <li th:each="memo : ${memos}" class="memo-item" th:dat
                <div>
                    <span class="title" th:text="${memo.title}"></
                    <br/>
                    <small th:text="${memo.description}"></small>
                </div>
                <span class="deadline" th:text="${memo.deadline}">
            </li>
        </ul>
        <a href="/memo/add" class="add-btn">＋ 添加任务</a>
    </div>
    <div class="learn-notes-column" id="notesSection">
        <h3>笔记</h3>
        <ul id="noteList">
            <li th:each="n : ${notes}">
                <span th:text="'[' + (n.type ?: 'SCRATCH') + '] '
            </li>
        </ul>
        <button type="button" id="openNoteModal" class="add-btn">＋
```

```
    </div>
</div>

<!-- 规划模式：高信息密度，目标/项目/全局 -->
<div th:if="${mindsetMode=='plan'}">
    <!-- 统计信息（如果用户偏好允许） -->
    <div class="statistics-section" th:if="${showStatistics != nul
        <h3>统计</h3>
        <div class="stat-item">
            <span>总任务数：</span><strong th:text="${totalTasks}"
        </div>
        <div class="stat-item">
            <span>已完成：</span><strong th:text="${doneTasks}"></
        </div>
        <div class="stat-item">
            <span>完成率：</span><strong th:text="${completionRate
        </div>
    </div>

    <!-- 规划完成得分曲线（Insight 插件，可由偏好关闭） -->
    <div class="score-section" th:if="${showScoreSection != null a
        <h3>规划完成得分曲线</h3>
        <div class="score-range">
            <input type="date" id="scoreStart">
            <input type="date" id="scoreEnd">
            <button type="button" id="scoreRefreshBtn">刷新</butto
        </div>
        <canvas id="scoreChart" height="120"></canvas>
        <div style="margin-top: 8px;">
            <button type="button" id="scoreSummaryBtn" class="add-
        </div>
        <div id="scoreSummary" class="score-summary" style="margin
    </div>

    <!-- 时间层视图切换 -->
    <div class="task-view-switch">
        <a th:href="@{/dashboard(taskView='today')}"
           th:classappend="${taskView=='today'} ? ' active' : ''">
        <a th:href="@{/dashboard(taskView='slot')}"
           th:classappend="${taskView=='slot'} ? ' active' : ''">时
        <a th:href="@{/dashboard(taskView='energy')}"
           th:classappend="${taskView=='energy'} ? ' active' : ''"
    </div>
    <form th:action="@{/memo/search}" method="get" class="filter-w
        <input name="keyword" placeholder="关键字">
        <input type="date" name="start">
        <input type="date" name="end">
```

```
    <button type="submit">搜索</button>
</form>

<!-- 模糊任务存在天数提示 -->
<div class="fuzzy-hint" th:if="${fuzzyNeedSplit != null and !f
    <h3>需要拆分的模糊任务</h3>
    <ul>
        <li th:each="t : ${fuzzyNeedSplit}">
            <span th:text="${t.title}"></span>
            <small>
                <span>已存在 </span>
                <span th:text="${T(java.time.temporal.ChronoUn
                <span> 天，考虑拆分为更小步骤</span>
            </small>
        </li>
    </ul>
</div>

<!-- 目标管理（仅在规划模式显示） -->
<div class="goals-section" id="goalsSection" th:if="${showGoal
    <h3>我的目标</h3>
    <button type="button" id="openGoalModal" class="add-btn">+
    <ul id="goalList">
        <li th:each="g : ${goals}">
            <span th:text="${g.name}"></span>
            <span class="goal-type" th:if="${g.goalType != nul
            <button type="button" class="goal-archive-btn" th:
            <button type="button" class="goal-delete-btn" th:d
        </li>
    </ul>
</div>

<!-- 今日视图：单列表 -->
<div th:if="${taskView=='today'}">
    <h3>今日可行动任务</h3>
    <ul>
        <li th:each="memo : ${memos}" class="memo-item" th:dat
            <div>
                <span class="title" th:text="${memo.title}"></
                <span class="goal-tags" th:if="${taskToGoals !
                    <span th:each="g : ${taskToGoals[memo.id]}
                </span>
                <br/>
                <small th:text="${memo.description}"></small>
                <div class="task-meta">
                    <span th:if="${memo.preferredSlot != null}
                          th:text="${memo.preferredSlot}"></sp
```

```
                    <span th:if="${memo.energyRequirement != n
                          th:text="${memo.energyRequirement}">
                </div>
            </div>
            <span class="deadline" th:text="${memo.deadline}">
        </li>
    </ul>
</div>

<!-- 时间段视图：上午 / 下午 / 晚上 -->
<div th:if="${taskView=='slot'}" class="slot-view">
    <div class="slot-column">
        <h3>上午</h3>
        <ul>
            <li th:each="memo : ${morningTasks}" class="memo-i
                <div>
                    <span class="title" th:text="${memo.title}
                    <br/>
                    <small th:text="${memo.description}"></sma
                </div>
                <span class="deadline" th:text="${memo.deadlin
            </li>
        </ul>
    </div>
    <div class="slot-column">
        <h3>下午</h3>
        <ul>
            <li th:each="memo : ${afternoonTasks}" class="memo
                <div>
                    <span class="title" th:text="${memo.title}
                    <br/>
                    <small th:text="${memo.description}"></sma
                </div>
                <span class="deadline" th:text="${memo.deadlin
            </li>
        </ul>
    </div>
    <div class="slot-column">
        <h3>晚上</h3>
        <ul>
            <li th:each="memo : ${eveningTasks}" class="memo-i
                <div>
                    <span class="title" th:text="${memo.title}
                    <br/>
                    <small th:text="${memo.description}"></sma
                </div>
                <span class="deadline" th:text="${memo.deadlin
```

```
            </li>
        </ul>
    </div>
    <div class="slot-column">
        <h3>未指定时间段</h3>
        <ul>
            <li th:each="memo : ${otherSlotTasks}" class="memo
                <div>
                    <span class="title" th:text="${memo.title}
                    <br/>
                    <small th:text="${memo.description}"></sma
                </div>
                <span class="deadline" th:text="${memo.deadlin
            </li>
        </ul>
    </div>
</div>

<!-- 精力视图：高 / 中 / 低 / 未指定 -->
<div th:if="${taskView=='energy'}" class="energy-view">
    <div class="slot-column">
        <h3>高精力</h3>
        <ul>
            <li th:each="memo : ${highEnergyTasks}" class="mem
                <div>
                    <span class="title" th:text="${memo.title}
                    <br/>
                    <small th:text="${memo.description}"></sma
                </div>
                <span class="deadline" th:text="${memo.deadlin
            </li>
        </ul>
    </div>
    <div class="slot-column">
        <h3>中精力</h3>
        <ul>
            <li th:each="memo : ${mediumEnergyTasks}" class="m
                <div>
                    <span class="title" th:text="${memo.title}
                    <br/>
                    <small th:text="${memo.description}"></sma
                </div>
                <span class="deadline" th:text="${memo.deadlin
            </li>
        </ul>
    </div>
    <div class="slot-column">
```

```
            <h3>低精力</h3>
            <ul>
                <li th:each="memo : ${lowEnergyTasks}" class="memo
                    <div>
                        <span class="title" th:text="${memo.title}
                        <br/>
                        <small th:text="${memo.description}"></sma
                    </div>
                    <span class="deadline" th:text="${memo.deadlin
                </li>
            </ul>
        </div>
        <div class="slot-column">
            <h3>未指定精力</h3>
            <ul>
                <li th:each="memo : ${otherEnergyTasks}" class="me
                    <div>
                        <span class="title" th:text="${memo.title}
                        <br/>
                        <small th:text="${memo.description}"></sma
                    </div>
                    <span class="deadline" th:text="${memo.deadlin
                </li>
            </ul>
        </div>
    </div>
    <a href="/memo/add" class="add-btn">＋ 添加任务</a>

    <!-- 归档 / 搁置区：不在默认视图中打扰用户，只在此折叠浏览 -->
    <div class="archived-section" th:if="${archivedOrShelved != nu
        <h3>归档 / 搁置任务</h3>
        <ul>
            <li th:each="t : ${archivedOrShelved}">
                <div>
                    <span class="title" th:text="${t.title}"></spa
                    <span class="status-tag" th:text="${t.status}"
                    <br/>
                    <small th:text="${t.description}"></small>
                </div>
                <span class="deadline" th:text="${t.deadline}"></s
            </li>
        </ul>
    </div>
</div>

</div>
```

```
</div>

<!-- Note Modal（学习模式和规划模式可用） -->
<div th:if="${mindsetMode=='learn' or mindsetMode=='plan'}" id="noteMo
    <div class="modal-content">
        <h3>创建笔记</h3>

        <!-- 笔记类型切换 -->
        <div class="note-type-switch">
            <button type="button" data-type="SCRATCH" class="note-type
            <button type="button" data-type="LEARNING" class="note-typ
            <button type="button" data-type="PROJECT" class="note-type
            <button type="button" data-type="RETROSPECTIVE" class="not
        </div>

        <!-- 通用标题 -->
        <input type="text" id="noteTitle" placeholder="标题（可选）">

        <!-- SCRATCH: 快速输入 -->
        <div id="notePanel-SCRATCH" class="note-panel">
            <textarea id="scratchContent" placeholder="随手记下你的想法
        </div>

        <!-- LEARNING: 概念 / 定义 / 示例 -->
        <div id="notePanel-LEARNING" class="note-panel" style="display
            <input type="text" id="learningConcept" placeholder="概念
            <textarea id="learningDefinition" placeholder="定义 / 关键
            <textarea id="learningExample" placeholder="示例 / 推导（可
        </div>

        <!-- PROJECT: 决策 / TODO / 问题 -->
        <div id="notePanel-PROJECT" class="note-panel" style="display:
            <textarea id="projectContext" placeholder="项目背景 / 决策"
            <textarea id="projectTodos" placeholder="TODO 列表（每行一
            <textarea id="projectIssues" placeholder="问题 / 风险（可选
        </div>

        <!-- RETROSPECTIVE: 失败/成功 / 原因 / 教训 -->
        <div id="notePanel-RETROSPECTIVE" class="note-panel" style="di
            <textarea id="retroWhat" placeholder="发生了什么？（成功 /
            <textarea id="retroWhy" placeholder="原因分析"></textarea>
            <textarea id="retroLessons" placeholder="教训 / 改进行动"><
        </div>

        <div class="modal-actions">
            <button type="button" id="saveNoteBtn">保存</button>
            <button type="button" id="closeNoteModal">取消</button>
```

```
        </div>
    </div>
</div>

<!-- 目标添加 Modal（仅规划模式） -->
<div th:if="${mindsetMode=='plan'}" id="goalModal" class="modal" style
    <div class="modal-content">
        <h3>添加目标</h3>
        <input type="text" id="goalName" placeholder="目标名称">
        <select id="goalType">
            <option value="">不指定类型</option>
            <option value="LONG_TERM">长期（季度/学期）</option>
            <option value="MID_TERM">中期（项目）</option>
            <option value="SHORT_TERM">短期</option>
        </select>
        <div class="modal-actions">
            <button type="button" id="saveGoalBtn">保存</button>
            <button type="button" id="closeGoalModal">取消</button>
        </div>
    </div>
</div>

<!-- 目标删除确认 Modal（仅规划模式） -->
<div th:if="${mindsetMode=='plan'}" id="goalDeleteModal" class="modal"
    <div class="modal-content">
        <h3>删除目标</h3>
        <p class="goal-delete-message">是否将绑定到该目标的任务一并删除？
        <div class="goal-delete-subtext">删除目标后无法恢复，任务可选择
        <div class="modal-actions">
            <button type="button" id="keepTasksBtn">仅删除目标</button
            <button type="button" id="deleteTasksBtn">删除目标和任务</b
            <button type="button" id="cancelDeleteGoalBtn">取消</butto
        </div>
    </div>
</div>

<!-- 右键菜单 -->
<div id="contextMenu" class="context-menu">
    <div class="context-menu-item" id="completeBtn">
        <span class="menu-icon">✓</span>
        <span>完成</span>
    </div>
    <div class="context-menu-item" id="shelveBtn">
        <span class="menu-icon">📦</span>
        <span>搁置</span>
    </div>
    <div class="context-menu-item" id="deleteBtn">
```

```
        <span class="menu-icon">🗑️</span>
        <span>删除</span>
    </div>
    <div class="context-menu-item" id="cancelBtn">
        <span class="menu-icon">❌</span>
        <span>取消</span>
    </div>
</div>

</body>
</html>
// ===== File: src/main/resources/templates/login.html =====
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Login</title>
    <link rel="stylesheet" href="/css/login.css">
</head>
<body>
<div class="login-container">
    <h2>Welcome Back</h2>
    <form action="/login" method="post">
        <input type="text" name="username" placeholder="Username" requ
        <input type="password" name="password" placeholder="Password"
        <input type="hidden" th:name="${_csrf.parameterName}" th:value
        <button type="submit" class="login-btn">Login</button>
        <button type="button" class="register-btn" onclick="window.loc
    </form>
</div>
</body>
</html>
// ===== File: src/main/resources/templates/preferenceSettings.html ==
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org" lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="_csrf" th:content="${_csrf.token}">
    <meta name="_csrf_header" th:content="${_csrf.headerName}">
    <title>偏好设置</title>
    <link rel="stylesheet" th:href="@{/css/dashboard.css}">
</head>
<body class="preference-settings-page" th:classappend="${theme == 'dar
<div class="container">
    <div class="preference-header">
        <h2>偏好设置</h2>
        <button type="button" class="reset-btn" id="resetPreferenceBtn
```

```
</div>
<p class="preference-intro">先选择要设置的思维模式，再修改该模式下可

<form th:action="@{/preference/update}" method="post" id="preferen
    <!-- 通用：与具体模式无关 -->
    <section class="preference-section preference-section-common">
        <h3>通用</h3>
        <div class="preference-item">
            <label for="theme">背景亮/暗</label>
            <select id="theme" name="theme">
                <option value="light" th:selected="${preference.th
                <option value="dark" th:selected="${preference.the
            </select>
        </div>
        <p class="preference-desc">切换思维模式或首次选择时，选择页
        <div class="preference-item">
            <label for="defaultMindsetMode">默认思维模式</label>
            <select id="defaultMindsetMode" name="defaultMindsetMo
                <option value="">不指定（每次手动选择）</option>
                <option value="EXECUTE" th:selected="${preference.
                <option value="LEARN" th:selected="${preference.de
                <option value="PLAN" th:selected="${preference.def
            </select>
        </div>
    </section>

    <!-- 选择要设置哪个思维模式的偏好 -->
    <section class="preference-section">
        <h3>选择要设置的思维模式</h3>
        <div class="preference-mode-tabs" role="tablist">
            <button type="button" class="mode-tab" role="tab" data
            <button type="button" class="mode-tab" role="tab" data
            <button type="button" class="mode-tab" role="tab" data
        </div>

        <div class="preference-mode-panel">
            <!-- 每屏任务数：执行、学习、规划均可用 -->
            <div class="preference-row" data-modes="execute learn
                <div class="preference-item">
                    <label for="maxVisibleTasks">每屏最多显示任务数
                    <input type="number" id="maxVisibleTasks" name
                           th:value="${preference.maxVisibleTasks}
                </div>
            </div>
            <!-- 显示未来任务：学习、规划 -->
            <div class="preference-row" data-modes="learn plan">
                <div class="preference-item">
```

```
        <label>
            <input type="checkbox" name="showFutureTas
                   th:checked="${preference.showFuture
            显示未来任务
        </label>
    </div>
</div>
<!-- 以下仅规划模式 -->
<div class="preference-row" data-modes="plan">
    <div class="preference-item">
        <label for="defaultTaskView">进入规划模式时默认
        <select id="defaultTaskView" name="defaultTask
            <option value="" th:selected="${preference
            <option value="slot" th:selected="${prefer
            <option value="energy" th:selected="${pref
        </select>
    </div>
</div>
<div class="preference-row" data-modes="plan">
    <div class="preference-item">
        <label>
            <input type="checkbox" name="showStatistic
                   th:checked="${preference.showStatis
            显示统计信息（完成率等）
        </label>
    </div>
</div>
<div class="preference-row" data-modes="plan">
    <div class="preference-item">
        <label>
            <input type="checkbox" name="showGoals" va
                   th:checked="${preference.showGoals
            显示目标管理区块
        </label>
    </div>
</div>
<div class="preference-row" data-modes="plan">
    <div class="preference-item">
        <label>
            <input type="checkbox" name="showScoreSect
                   th:checked="${preference.showScoreS
            显示规划完成得分曲线
        </label>
    </div>
</div>
<div class="preference-row" data-modes="plan">
    <div class="preference-item">
```

```
                        <label>
                            <input type="checkbox" name="showFuzzyHint
                                   th:checked="${preference.showFuzzyH
                            显示「需要拆分的模糊任务」提示
                        </label>
                    </div>
                </div>
                <div class="preference-row" data-modes="plan">
                    <div class="preference-item">
                        <label>
                            <input type="checkbox" name="showArchivedS
                                   th:checked="${preference.showArchiv
                            显示归档/搁置任务区块
                        </label>
                    </div>
                </div>
                <div class="preference-row" data-modes="plan">
                    <div class="preference-item">
                        <label for="fuzzyTaskDaysThreshold">模糊任务存
                        <input type="number" id="fuzzyTaskDaysThreshol
                               th:value="${preference.fuzzyTaskDaysThr
                    </div>
                </div>
            </div>
        </section>

        <input type="hidden" th:name="${_csrf.parameterName}" th:value
        <div class="preference-actions">
            <button type="submit" class="add-btn">保存设置</button>
            <a href="/dashboard" class="add-btn">返回</a>
        </div>
    </form>
</div>
<script>
(function() {
    var page = document.querySelector('.preference-settings-page');
    var selectedMode = (page && page.getAttribute('data-selected-mode'
    var tabs = document.querySelectorAll('.mode-tab');
    var rows = document.querySelectorAll('.preference-row[data-modes]'

    function showForMode(mode) {
        selectedMode = mode;
        tabs.forEach(function(t) {
            var isActive = t.getAttribute('data-mode') === mode;
            t.classList.toggle('active', isActive);
            t.setAttribute('aria-selected', isActive ? 'true' : 'false
        });
```

```
        rows.forEach(function(row) {
            var modes = (row.getAttribute('data-modes') || '').split(/
            row.style.display = modes.indexOf(mode) !== -1 ? '' : 'non
        });
    }

    tabs.forEach(function(t) {
        t.addEventListener('click', function() {
            showForMode(this.getAttribute('data-mode'));
        });
    });

    showForMode(selectedMode);

    // 恢复默认设置：只重置当前页面表单的值，不提交到后端
    var resetBtn = document.getElementById('resetPreferenceBtn');
    if (resetBtn) {
        resetBtn.addEventListener('click', function() {
            var theme = document.getElementById('theme');
            var defaultMindsetMode = document.getElementById('defaultM
            var maxVisibleTasks = document.getElementById('maxVisibleT
            var showFutureTasks = document.querySelector('input[name="
            var showStatistics = document.querySelector('input[name="s
            var showGoals = document.querySelector('input[name="showGo
            var showScoreSection = document.querySelector('input[name=
            var showFuzzyHint = document.querySelector('input[name="sh
            var showArchivedSection = document.querySelector('input[na
            var defaultTaskView = document.getElementById('defaultTask
            var fuzzyTaskDaysThreshold = document.getElementById('fuzz

            if (theme) theme.value = 'light';
            if (defaultMindsetMode) defaultMindsetMode.value = '';
            if (maxVisibleTasks) maxVisibleTasks.value = '';

            if (showFutureTasks) showFutureTasks.checked = true;
            if (showStatistics) showStatistics.checked = true;
            if (showGoals) showGoals.checked = true;
            if (showScoreSection) showScoreSection.checked = true;
            if (showFuzzyHint) showFuzzyHint.checked = true;
            if (showArchivedSection) showArchivedSection.checked = tru

            if (defaultTaskView) defaultTaskView.value = '';
            if (fuzzyTaskDaysThreshold) fuzzyTaskDaysThreshold.value =
        });
    }
})();
</script>
```

```
</body>
</html>
// ===== File: src/main/resources/templates/register.html =====
<!DOCTYPE html>
<!-- 引入th名称空间 -->
<html lang="en" xmlns:th="http://www.thymeleaf.org">

<head>
    <meta charset="UTF-8">
    <title>Register</title>
    <link rel="stylesheet" href="/css/register.css">
</head>

<body>
<div class="register-container">
    <h2>Register</h2>
    <form action="/register" method="post">
        <input type="text" name="username" placeholder="Username" requ
        <input type="password" name="password" placeholder="Password"
        <!-- 隐式添加csrf token, 避免钓鱼链接的csrf攻击 -->
        <input type="hidden" th:name="${_csrf.parameterName}" th:value
        <button type="submit" class="register-btn">Register</button>
        <button type="button" class="login-btn" onclick="window.locati
    </form>

    <div style="color: red;" th:if="${errorMessage}">
        <span th:text="${errorMessage}"></span>
    </div>
</div>
</body>

</html>
// ===== File: src/main/resources/templates/selectFeatures.html =====
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org" lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="_csrf" th:content="${_csrf.token}">
    <meta name="_csrf_header" th:content="${_csrf.headerName}">
    <title>Select View Mode</title>
    <link rel="stylesheet" th:href="@{/css/selectFeatures.css}">
</head>
<body>
<div class="container">
    <h2>选择思维模式</h2>
    <form th:action="@{/select-features}" method="post">
        <input type="hidden" th:name="${_csrf.parameterName}" th:value
```

```
        <div class="feature-option">
            <input type="radio" id="execute" name="mindsetMode" value=
                   th:checked="${defaultMindsetMode == null ? true : d
            <label for="execute">
                <strong>执行模式</strong>
                <small>低信息密度，只看今日+当前任务，专注完成/搁置/下一个
            </label>
        </div>

        <div class="feature-option">
            <input type="radio" id="learn" name="mindsetMode" value="l
                   th:checked="${defaultMindsetMode != null and defaul
            <label for="learn">
                <strong>学习模式</strong>
                <small>中信息密度，任务+笔记并排，创建笔记、关联任务</sma
            </label>
        </div>

        <div class="feature-option">
            <input type="radio" id="plan" name="mindsetMode" value="pl
                   th:checked="${defaultMindsetMode != null and defaul
            <label for="plan">
                <strong>规划模式</strong>
                <small>高信息密度，目标/项目/全局，目标树、任务列表、统计
            </label>
        </div>

        <button type="submit" class="submit-btn">继续</button>
    </form>
</div>
</body>
</html>
// ===== File: src/test/java/org/zhzssp/memorandum/MemorandumApplicati
package org.zhzssp.memorandum;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class MemorandumApplicationTests {

    @Test
    void contextLoads() {
        System.out.println("Context loads successfully.");
    }
```

}